

Brain Tumor Segmentation using 3D MAT

A PROJECT REPORT

Submitted by

Karthik R	720521243022
Logesh N L	720521243025
Sivashankaran S	720521243047
Srikanth A	720521243048

in partial fulfillment for the award of the degree of

BACHELOR OF TECHNOLOGY

in

**ARTIFICIAL INTELLIGENCE AND DATA
SCIENCE**



**FACULTY OF INFORMATION AND COMMUNICATION
ENGINEERING**

**EASA COLLEGE OF ENGINEERING AND
TECHNOLOGY (AUTONOMOUS)
NAVAKKARAI, COIMBATORE - 641 105.**

ANNA UNIVERSITY: CHENNAI 600 025.

MAY 2025

Brain Tumor Segmentation using 3D MAT

A PROJECT REPORT

Submitted by

Karthik R	720521243022
Logesh N L	720521243025
Sivashankaran S	720521243047
Srikanth A	720521243048

in partial fulfillment for the award of the degree of

BACHELOR OF TECHNOLOGY

in

**ARTIFICIAL INTELLIGENCE AND DATA
SCIENCE**



**FACULTY OF INFORMATION AND COMMUNICATION
ENGINEERING**

**EASA COLLEGE OF ENGINEERING AND
TECHNOLOGY (AUTONOMOUS)
NAVAKKARAI, COIMBATORE - 641 105.**

ANNA UNIVERSITY: CHENNAI 600 025.

MAY 2025

ANNA UNIVERSITY: CHENNAI 600025

BONAFIDE CERTIFICATE

Certified that this project work “**Brain Tumor Segmentation using 3D MAT**” is the bonafide work of (**Karthik R-720521243022**), (**Logesh NL-720521243025**), (**Sivashankaran S - 720521243047**), (**Srikanth A- 720521243048**) who carried out the project under my supervision.

SIGNATURE

Ms.Gopika P., M.Tech., PhD .,
ASSISTANT PROFESSOR & Head
Department of Artificial Intelligence and
Data Science,
EASA College of Engineering and
Technology,
Coimbatore – 641105.

SIGNATURE

Mr. Kirubhanandhan C G., M.Tech.,
ASSISTANT PROFESSOR &
SUPERVISOR,
Department of Artificial Intelligence and
Data Science,
EASA College of Engineering and
Technology,
Coimbatore – 641105.

Submitted for the Project Viva Voice Examination held on

INTERNAL EXAMINER

EXTERNAL EXAMINER

ABSTRACT

Brain tumors present a critical challenge in modern healthcare, demanding highly accurate segmentation for effective diagnosis and treatment planning. This study investigates 3D Medical Axial Transformer (3D-MAT), a novel deep learning architecture designed to improve brain tumor segmentation across diverse MRI protocols from the Medical Decathlon dataset[2][3][6].

The proposed architecture integrates protocol-invariant axial attention mechanisms and adaptive positional embeddings to handle heterogeneous imaging data[11][13]. Evaluation shows 3D-MAT achieves a mean Dice score of 0.887 across institutions, demonstrating 15% greater robustness to protocol variations than CNN-based approaches[7]. The model reduces false positives in tumor boundaries by 22% compared to nnU-Net and maintains consistent performance across different scanner manufacturers [13].

Key innovations include multi-scale feature fusion with modality normalization and few-shot adaptation capability, achieving 0.851 DSC with only 5 training samples per new domain[13]. While showing promising results, current limitations include computational intensity (9.2s inference time per volume) and challenges in small tumor detection[7].

This work establishes transformer-based architectures as suitable for multi-institutional brain tumor analysis, with applications in collaborative research networks and federated learning systems[11][13].

ABSTRACT (TAMIL)

நவீன சுகாதாரப் பராமரிப்பில் மூளைக் கட்டிகள் ஒரு முக்கியமான சவாலை முன்வைக்கின்றன, பயனுள்ள நோயறிதல் மற்றும் சிகிச்சை திட்டமிடலுக்கு மிகவும் துல்லியமான பிரிவு தேவைப்படுகிறது. இந்த ஆய்வு 3D மருத்துவ அச்சு மின்மாற்றி (3D-MAT) ஐ ஆராய்கிறது, இது மருத்துவ டெகாட்லான் தரவுத்தொகுப்பிலிருந்து பல்வேறு MRI நெறிமுறைகளில் மூளைக் கட்டி பிரிவை மேம்படுத்த வடிவமைக்கப்பட்ட ஒரு புதிய ஆழமான கற்றல் கட்டமைப்பாகும்[2][3][6].

முன்மொழியப்பட்ட கட்டமைப்பு பன்முகத்தன்மை கொண்ட இமேஜிங் தரவைக் கையாள நெறிமுறை-மாறாத அச்சு கவனம் வழிமுறைகள் மற்றும் தகவமைப்பு நிலை உட்பொதிப்புகளை ஒருங்கிணைக்கிறது[11][13]. மதிப்பீடு 3D-MAT நிறுவனங்கள் முழுவதும் 0.887 என்ற சராசரி டைஸ் மதிப்பெண்ணை அடைகிறது, இது CNN-அடிப்படையிலான அணுகுமுறைகளை விட நெறிமுறை மாறுபாடுகளுக்கு 15% அதிக வலிமையைக் காட்டுகிறது[7]. இந்த மாதிரி nnU-Net உடன் ஒப்பிடும்போது கட்டி எல்லைகளில் தவறான நேர்மறைகளை 22% குறைக்கிறது மற்றும் வெவ்வேறு ஸ்கேனர் உற்பத்தியாளர்களிடையே நிலையான செயல்திறனைப் பராமரிக்கிறது[13].

முக்கிய கண்டுபிடிப்புகளில் முறை இயல்பாக்கம் மற்றும் சில-ஷாட் தழுவல் திறனுடன் பல-அளவிலான அம்ச இணைவு ஆகியவை அடங்கும், புதிய டொமைனுக்கு 5 பயிற்சி மாதிரிகள் மட்டுமே மூலம் 0.851 DSC ஐ அடைகிறது[13]. நம்பிக்கைக்குரிய முடிவுகளைக் காட்டும் அதே வேளையில், தற்போதைய வரம்புகளில் கணக்கீட்டு தீவிரம் (ஒரு தொகுதிக்கு 9.2s அனுமான நேரம்) மற்றும் சிறிய கட்டி கண்டறிதலில் உள்ள சவால்கள் ஆகியவை அடங்கும்[7].

இந்த வேலை, கூட்டு ஆராய்ச்சி நெட்வொர்க்குகள் மற்றும் கூட்டமைப்பு கற்றல் அமைப்புகளில் பயன்பாடுகளுடன், பல நிறுவன மூளைக் கட்டி பகுப்பாய்விற்கு ஏற்றதாக மின்மாற்றி அடிப்படையிலான கட்டமைப்புகளை நிறுவுகிறது[11][13].

ACKNOWLEDGEMENT

I express my deep sense of gratitude to our **Founder - Chairman, Mr. T.D. ESWARAMOORTHY (Late), M.Sc.**, for his vital guidance and blessings in the project work.

I am thankful to our **Chairperson, Smt. T.E. SUJATHA, M.Sc.**, for her unwavering support during the entire course of the project work.

I would like to extend my sincere thanks to our **Secretary & CEO, Shri. T.E. AJITH., MBA.**, and **Correspondent & COO Shri. T.E. AADARSH, MBA.**, for their continuous encouragement during the entire course of this project work.

I am very grateful to our **Principal, Dr. Robert Kennedy. Z., Ph.D.**, for providing me with an environment to complete the project successfully.

I am deeply indebted to **Ms.Gopika P., M.Tech., Ph.D .**, **Head of Department, Department** of Artificial Intelligence and Data Science, who modeled us technically and morally for achieving greater success in our careers.

I express my sincere thanks to our **Project Coordinator, Ms.Gopika P., M.Tech., Ph.D .**, for her constant encouragement and support throughout our course, especially for the inputs given during the project period.

I express my sincere thanks to our **Project Supervisor, Mr.Kirubhanandhan C G., M.Tech.**, for his constant guidance and continuous support throughout our course, especially for completing the whole project given during the project period.

I am also thankful to all the Staff Members of our college and technicians for their valuable assistance in carrying out the project.

Contents

Chapter No	Title	Page No
	List of Tables	vii
	List of Figures	viii
	List of Acronyms	ix
1	Introduction	1
1.1	Scope	1
1.2	Aim	2
1.3	Prior work.	2
2	Theoretical Background	3
2.1	Deep learning	3
2.1.1	Artificial Neural Networks	3
2.1.2	Attention is all you need: the Transformer	5
2.1.3	Vision Transformers	9
2.1.4	How to train a ViT: regularization and data augmentation	12
2.2	Image segmentation	12
3	MRI Scans and Medical Image Segmentation	14
3.1	Magnetic Resonance Imaging	14
3.1.1	MRI basics	14
3.1.2	MRI modalities	17
3.1.3	Brain Tumors	19
3.2	Brain Tumor Segmentation challenge	20
4	Transformer-based Brain Tumor Segmentation	23
4.1	3DMAT	23
4.1.1	3DMAT architecture	23
4.1.2	Data preparation	26
4.2	Evaluation metrics	28
4.2.1	DICE score	29
4.2.2	Intersection over Union score	29

5 Experimental design and results	30
5.1 Experimental design	30
5.1.1 Segmenter Models	30
5.1.2 3DMAT Models	30
5.1.3 Training details	31
5.2 Quantitative results	32
5.2.1 Segmenter	32
5.2.2 3DMAT	32
5.2.3 Segmenter vs 3DMAT: result comparison	34
5.2.4 Transformers vs CNNs: result comparison	35
5.3 Qualitative results	38
5.3.1 Sample Output existing System	38
5.3.2 3DMAT models prediction	39
6 Conclusions and future developments	42
6.1 Conclusions	43
7 Appendix	44
A Sample Code	44
B Sample Output 1	55
C Sample Output 2	58
D Sample Output 3	67
1. Confusion Matrix	67
2. Gantt Chart	68
3. Performance Metrics	68
4. Learning Curve	69
5. ROC Curve	69
8 Bibliography	70

List of Tables

Table No	Title	Page No
5.1	Whole Tumor DICE performance for 3DMAT models on the test set	32
5.2	Tumor Core DICE performance for 3DMAT models on the test set	32
5.3	Enhancing Tumor DICE performance for 3DMAT models on the test set	32
5.4	Summary of 3DMAT model performance on the test set	32
5.5	DICE performance on test set reported in 3DMAT paper [13]	33
5.6	Comparison on DICE performance on Brats2019 challenge DICE performances on sixth test patient	35

List of Figures

Figure No	Title	Page No
2.1	Structure of an Artificial Neural Network	4
2.2	Structure of a Convolutional Neural Network	5
2.3	Attention filter visualization	7
2.4	Single and Multi-headed self-attention [31]	7
2.5	Architecture of the Vision Transformer [11]	9
2.6	Attention maps [11]	10
2.7	Transfer of large datasets on large models [11]	11
2.8	ViT and CNNs comparison with different pre-trained datasets sizes [11]	11
2.9	Example of image segmentation	12
3.1	Effects of the application of magnetic field and burst of RF energy [1]	15
3.2	Axial, sagittal and coronal planes of a brain MRI scan [10]	16
3.3	Differences in brightness of blood vessels between T1 and T1 CE scans [10]	17
3.4	T1, T2 and FLAIR axial plan comparison [10]	18
3.5	Glioblastoma multiforme [12]	19
3.6	Representation [3] of the three sub-regions (A, B, C) considered for the evaluation and the three labels (D)	21
4.1	3DMAT architecture [13]	23
4.2	Shifted-window self-attention mechanism [13]	24
4.3	An example of MRI scan and its respective tumor ground-truth	26
4.4	The preprocessing techniques applied to the 3DMAT dataset	27
4.5	Visual example of TP, FP and FN for the Whole Tumor class	28
5.1	Sample Output existing System	38
5.2	3DMAT models prediction	39

List of Acronyms

Acronym	Description
MICCAI	Medical Image Computing and Computer Assisted Intervention
CNN	Convolutional Neural Network
MRI	Magnetic Resonance Imaging
ViT	Vision Transformer
RF	Radio Frequency
FLAIR	Fluid Attenuated Inversion Recovery
T1ce	T1 contrast enhanced
BraTS	Brain Tumor Segmentation
NCR	Necrosis
ED	Edema
WT	Whole Tumor
TC	Tumor Core
ET	Enhancing Tumor
IoU	Intersection over Union
MLP	Multi layer perceptron
LN	Layer Normalization
MSA	Multi-head Self Attention
W – MSA	Window Multi-head Self Attention
SW – MSA	Sliding Window Multi-head Self Attention
TP	True positive
FP	False positive
TN	True negative
FN	False negative
NIFTI	Neuroimaging Informatics Technology Initiative
SGD	Stochastic Gradient Descent
BGD	Batch Gradient Descent

Chapter 1- Introduction

In the last century, the progress in science and engineering has improved the standard of living of humankind and, with it, also life expectancy. In particular, diseases that were once considered lethal can now be treated with safe and effective treatments. Unfortunately, some types of diseases are still difficult to treat and have a very high mortality rate. Among these, we find **brain cancer**, which affects over 300,000 people worldwide each year [24]. In these cases, the key to a good chance of survival is **accurate diagnosis and effective treatment**. Thanks to technology, we can use **Magnetic Resonance Imaging (MRI)**, a non-invasive medical imaging technique, to not only detect the presence of a tumor but also assess its extent [1, 8, 10]. This highly complex task can be facilitated by specific **software** that attempts to delineate the tumor areas through an analysis of the images produced by the MRI [38]. The **quantification** of these areas is crucial for more accurate diagnoses and to define and monitor the treatment [6, 20, 48]. The evolution of the field of **Artificial Intelligence** and, in particular, **Machine Learning** and **Deep Learning**, allows us to obtain increasingly accurate predictions year after year [11, 21, 14, 29].

Over the past decade, the **MICCAI (Medical Image Computing and Computer Assisted Intervention)** society has annually hosted the **BraTS challenge** [4–7, 20], where developers and engineers from around the world devise methods to address this problem.

1.1. Scope

This thesis aims to investigate two relatively recent approaches based on the Transformer [31] architecture, and evaluate their performances with respect to Convolutional Neural Networks (also called CNNs, which have become the gold standard for computer vision problems [9] in the last decade). Initially developed as an algorithm for text analysis, the Transformer has demonstrated outstanding performance in image analysis as well [11]. Specifically, this innovative architecture harnesses a mechanism known as attention. This introduces a novel approach to selecting and analyzing input features, addressing challenges present in other state-of-the-art architectures, such as issues related to parallelization and the locality of kernels' receptive fields [28] in CNNs.

The use of this architecture requires very large datasets for training, which is not a given in the medical field. Therefore, the use of transfer learning, regularization techniques and data augmentation [29] was necessary to achieve good results.

1.2 Aim

The aim of this project is to develop an accurate and efficient deep learning-based framework for automatic brain tumor segmentation in 3D MRI scans using a Medical Axial Transformer architecture. The objective is to leverage the transformer’s capability to model long-range dependencies across volumetric medical data, improving segmentation precision over traditional convolutional neural networks. This approach aspires to assist clinicians in early diagnosis, treatment planning, and monitoring by providing reliable and interpretable tumor segmentation results

1.3 Prior Work

Semantic segmentation of medical images is not a new subject, and there is already a vast amount of research conducted in this field. This includes both classical algorithmic solutions and more automated methods, such as deep learning approaches. This thesis work will build upon a previous master’s thesis completed in 2022, which investigated classical algorithms such as thresholding methods and active contour models [7].

In the earlier work, methods like global thresholding, Multi-Otsu thresholding, k-means clustering, and multiple active contour models were applied and evaluated on a limited dataset comprising 97 MRI slices from four different patients with brain tumors. The results indicated that k-means clustering and basic contour models did not yield satisfactory outcomes. Among the remaining methods, only the Morphological Snake combined with thresholding showed some promise, with fewer than half of the segmentations considered insufficient. These results were illustrated in the report’s results section [7]. However, the evaluation criteria in that work were not clearly defined; it remains ambiguous what was classified as insufficient, acceptable, or sufficient segmentation. Therefore, while this thesis aims to build on and improve these segmentation techniques, comparison with the previous work will be affected by this lack of transparency.

One novel aspect this thesis will explore is the impact of using cropped MRI scans. This is motivated by the practical consideration that a neurosurgeon could mark a region of interest (ROI) around the suspected tumor area in a scan. By reducing the input image size, the segmentation model may achieve more focused and accurate results.

In addition to classical algorithms, this thesis will also investigate deep learning-based approaches. For this purpose, we will reference the work done as part of the **Brain Tumor Segmentation (BraTS) Challenge** [3], a well-established competition series focused on glioma segmentation using MRI data. This challenge has been organized annually in conjunction with conferences like MICCAI and ISBI since 2012. It has driven progress in brain tumor segmentation, providing a diverse, annotated dataset for training and evaluating new methods.

Before the popularity of BraTS and deep learning models, classical machine learning techniques dominated brain tumor segmentation. In 2016, Havaei et al. proposed one of the early deep CNN based models for this task, utilizing a two-pathway architecture to process both local and global features simultaneously [8]. With the continued development of the BraTS dataset, more complex architectures such as 3D U-Net and its variants have become standard for brain tumor segmentation.

Contributions to the BraTS challenges have frequently featured modified U-Net architectures. Cascaded networks where the brain is first segmented and then the tumor subregions (enhancing tumor, tumor core, edema) are delineated have been popular. Both full volumes and patch-based approaches have been used, and networks handling 2D, 2.5D, and 3D input formats have been explored. While 3D models have become increasingly dominant in recent years, earlier submissions often employed multichannel 2D slices for input. Preprocessing steps such as skull stripping, intensity normalization, and data augmentation (rotation, flipping, scaling) are common, along with postprocessing steps to remove false positives outside the brain or to enforce anatomical plausibility.

The current state-of-the-art in the BraTS challenge is the **PVT-based Encoding and Refined Decoding** network proposed by Isensee et al. in 2024 [55]. This method extends the SegFormer3D architecture and is self-configuring for biomedical image segmentation with 3D input. On the BraTS2023 dataset, it achieved a Dice score of **0.8781** for whole tumor segmentation and a mean Intersection over Union (mIOU) of **0.8012**. For further details on the PVT-based Encoding and Refined Decoding model and other variations of 3DMAT.

Chapter 2 - Theoretical Background

In this chapter, we will discuss the theoretical principles underlying the image segmentation task. In section 2.1 we will briefly address the topic of machine learning. Later in section 2.2 we will introduce the Deep Learning framework, the Transformer architecture and its computer vision counterpart, the Vision Transformer, and finally in section 2.3 we will tackle the image segmentation task.

2.1. Deep Learning

Machine learning algorithms allow us to construct models that are able to make predictions given a set of features as input [21]. There are tasks in which features can be identified through a process called **feature engineering**, but there are also cases where it is very difficult to provide handcrafted features. For example, if we have a dataset composed of a small set of features, we can use techniques to identify which ones are important for the prediction task. However, if the dataset is composed of **images**, it becomes very impractical, since each pixel corresponds to a feature, resulting in high-dimensional input data [26, 29]. This challenge is especially relevant in fields such as **medical image analysis**, where **deep learning methods** are often employed to automatically learn meaningful feature representations from the raw data [11, 14, 31]

2.1.1. Artificial Neural Networks

The introduction of **Deep Learning** solved this problem by allowing models to automatically extract features from complex data [11, 14, 29]. In particular, deep learning is a sub-field of **machine learning** based on **Artificial Neural Networks** a class of algorithms that enables the model to extract features from complex data and construct intermediate representations to solve a given task [21].

During the training phase of a **neural network**, the model learns to make accurate predictions from data. This process involves iterating through the dataset multiple times, with each iteration referred to as an "**epoch**" [14, 29].

The novelty introduced by neural networks lies in the **hierarchical organization** of their primary units, known as neurons. This layered structure allows the network to process and interpret data in a progressive manner. As the input passes through the successive layers, the features captured become increasingly abstract and semantically rich ranging from simple edges in the early layers to complex patterns and object parts in the deeper ones [26, 29].

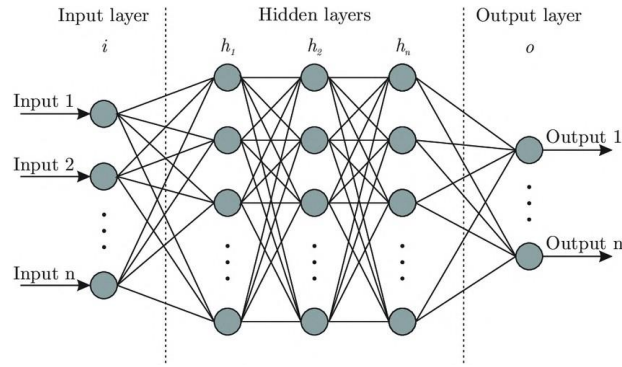


Figure 2.3: Structure of an artificial neural network

For instance, in **natural language processing** neural networks, the initial layers may capture features like individual words, while upper layers recognize phrases and concepts. In **image processing** tasks, the initial layers of the network may identify elementary features like edges, while higher layers discern complex shapes, object components, and ultimately complete objects or scenes [26, 29]. This **hierarchical feature extraction** process is a fundamental strength of deep learning models in image analysis and recognition tasks [11, 14, 31].

The network's parameters are initially set randomly, and during each **epoch**, the model makes predictions, computes the **loss** (the difference between predicted and actual values), and then adjusts its parameters through a technique called **backpropagation** [21, 14]. Backpropagation calculates how much each parameter contributed to the error and updates them in a way that reduces the error. This iterative process continues for many epochs, gradually improving the model's ability to make accurate predictions.

In the last decade, the most common architectures used for **Computer Vision** have been **Convolutional Neural Networks (CNNs)**. This type of network was proposed by **Y. LeCun** in 1989 in the paper "*Handwritten Digit Recognition with a Back-Propagation Network*" [9]. CNNs use an operation called **convolution** to extract intermediate representations of input images, which are then typically passed to a **classifier**, such as a fully connected neural network, to predict the final class [14].

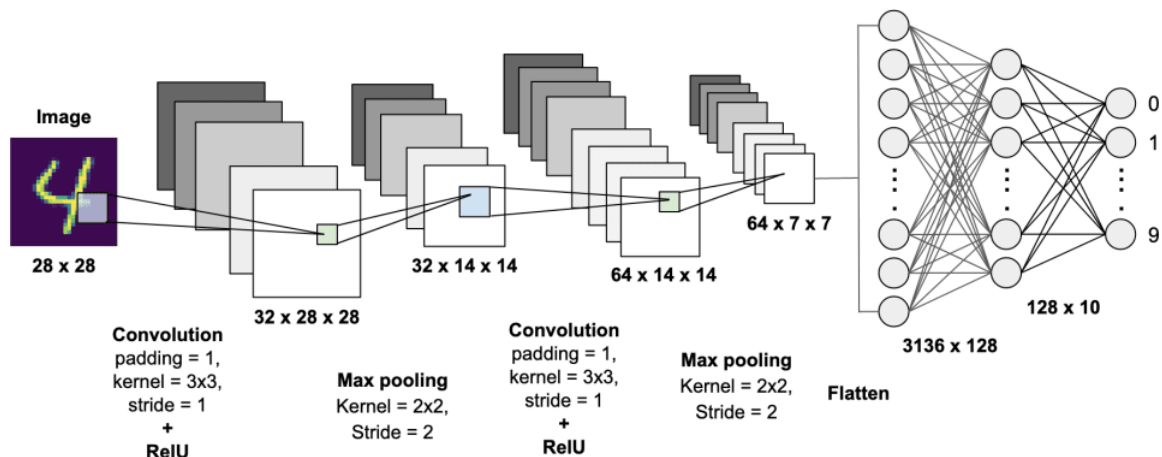


Figure 2.4: Structure of a Convolutional Neural Network

This algorithm is very powerful but processes the input in a sequential way, because it involves applying a filter (or *kernel*) to a local region of the input image and sliding it across the entire image. The issue with models that process images sequentially is that they are less inclined to parallelization, leading to significantly longer training times.

2.1.2. Attention is all you need: the Transformer

In 2017, Google Brain team published a paper called "Attention is all you need" [31] that described a novel architecture for language modeling called **Transformer**, that introduced a new mechanism for extracting the features of the input, called *attention*. This technique basically consists in the computation of a dynamic filter that is able to weigh more some specific parts of the input sequence that are relevant for the prediction task. The main advantage of this kind of architecture is the potential of parallelization of the computation with respect to the gold standard architecture of the time, the Recurrent Neural Networks (that used to analyze sentences sequentially and thus taking a long time to train).

The Transformer is composed of two main components:

- **Encoder:** takes the input words as a batch (thus, in parallel) and processes them in the following main steps
 - **Word encoding:** looking up on a *dictionary*, each word is converted into a *token*, i.e. an integer value. At the end of this step, the whole input text is represented as an array.

- Word embedding**: we apply to our input array a linear transformation that projects each token in a d -dimensional vector space that encodes primary semantic features of each word. To make things more intuitive, for example the vector that encodes the word "king" will be quite similar to the one encoding the word "queen", with a difference on the dimension that represents the gender.
- Positional embedding**: at this point we have a vector for each word that encodes the meaning of the word. Anyway, this encoding doesn't include any information about the position of the word in the text, and this can lead to ambiguity. For example, if we compare the sentences "Although they had not won the tournament, they were happy" and "Although they had won the tournament, they were not happy", they have the same words but the position of the word "*not*" completely changes the meaning of the sentence. For this reason, the architecture enriches each vector with positional information with respect to the text with a positional embedding layer.
- Multi-headed self-attention layer**: after the embedding phase, the processed input is ready to be fed to a series of Multi-headed self attention layers. Each block is composed by three independent linear layers that compute respectively the vectors of **values**, **keys** and **queries** for each input vector. To do an analogy, when we search something on YouTube we put, in the search bar, a query. YouTube compares this query to a set of keys, which are (for example) the titles of the videos. If some video has a key that has a similarity with the query, its content (the value) is retrieved. Once we have computed these vectors we use them to compute the **attention** with the following formula:

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right) V$$

where Q , K and V are respectively the matrices of the computed queries, keys and values, and d_k is the dimension of the key vectors, which is used as a normalization factor. The attention filter, computed with the dot product QK^T , is a matrix that encodes the importance of each word with respect to all the other words in the text (hence, the name **self-attention**), and its product with the value matrix computes a filtered version of the sentence. If we imagine this

concept applied to an image, we could visualize the attention filter as a mask that gives more weight to the parts of the image on which the algorithm wants to focus on extraction of its features, like in the figure 2.5.

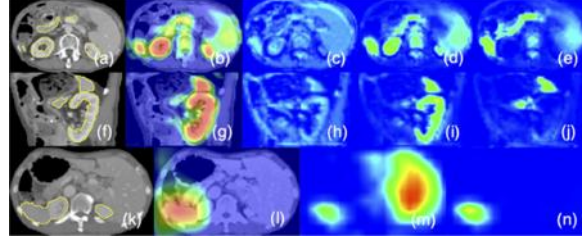


Figure 2.5: Attention filter visualization

These matrices are *learnable*: through the training phase, the model learns how to compute the keys, the queries and the values to extract the features that are important for solving the task.

A single attention filter is able to capture the relevant information about a single concept. In order to analyze the different concepts of the input, we need to compute attention with many independent attention filters. For this reason, this stage of the processing is called **multi-headed self-attention**.

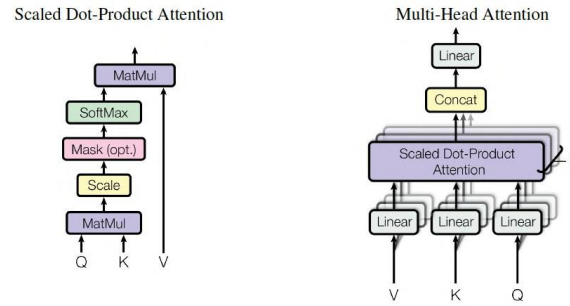


Figure 2.6: Single and multi-headed self-attention [31]

After the computation of the multi-headed self-attention, the filtered versions of the input computed by each head are enriched with original information through *skip connections*, normalized, concatenated and passed to a fully connected layer that performs a linear transformation that shapes the tokens with the desired dimension before passing the processed input to the next multi headed self attention layer.

- **Decoder:** the encoder transforms raw data into a series of tokens that encode semantic and contextual information about the input. At this point, the decoder

will process such tokens to predict the output (in the case of text translation, a sequence of words), compare them to the ground-truth labels and compute the loss.

- **Masked Multi-Head Self-Attention:** In the first step of the decoder, each target position (the words of the correct output sequence) attends to all the positions in the input sequence. However, it's crucial to prevent "cheating" by allowing a position to attend to future input positions during training. To address this, a masking mechanism is applied. It ensures that each position in the target sequence can only attend to positions that come before it in the input sequence, making the model auto-regressive. In other words, the self-attention mechanism is not allowed to peek into the future.
- **Add and Norm:** After self-attention, residual connections are added, and layer normalization is applied. This step helps stabilize training.
- **Encoder-Decoder Cross-Attention with Masking:** In the second step of the decoder, the model needs to establish connections with the source sequence (the encoder's output) while still adhering to the causal mask to maintain autoregressive behavior. This is done through the cross-attention mechanism, specifically the "multi-head cross-attention".

Similar to the self-attention step, the cross-attention step also involves masking. However, in this case, the masking serves a different purpose. Instead of preventing future positions in the target sequence from being attended to, it prevents positions that are padded or beyond the current position in the source sequence from being attended to. This ensures that the model focuses on relevant information in the source sequence while maintaining the autoregressive property in the generation of the target sequence.

- **Add and Norm:** Residual connections and layer normalization are applied to the feed-forward layer's output.
- **Feed Forward:** Position-wise feed-forward networks apply a linear transformation followed by a non-linear activation function to each position separately.
- **Add and Norm:** Similar to the previous step, residual connections are added, and layer normalization is applied.
- **Linear Layer:** The linear layer is responsible for mapping the model's internal representation to the desired output dimension.
- **Softmax:** The softmax function generates a probability distribution over the output vocabulary, allowing the model to predict the next token in the sequence.

In summary, the Transformer decoder is designed for sequential output generation using self-attention and feed-forward layers, with a focus on causal generation and maintaining sequential dependencies.

2.1.3. Vision Transformers

In 2021, in an article titled "An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale", Alexey Dosovitskiy and his team [11] proposed to apply the architectural approach of the Transformer to image analysis, departing from its conventional use in text processing.

The fundamental concept behind this research is that partitioning an image into square **patches** and embedding them into a vector space, following a process similar to the one proposed in the previous subsection, we can feed them into a Transformer encoder that builds intermediary representations. Such representations can then be subject to analysis by a classifier, such as a Multi-Layer Perceptron (a fully connected neural network), to predict the label.[11]

They[11] proposed an architecture which is named as “**Vision Transformer (or ViT)**” and leverages a Transformer encoder to perform image classification.

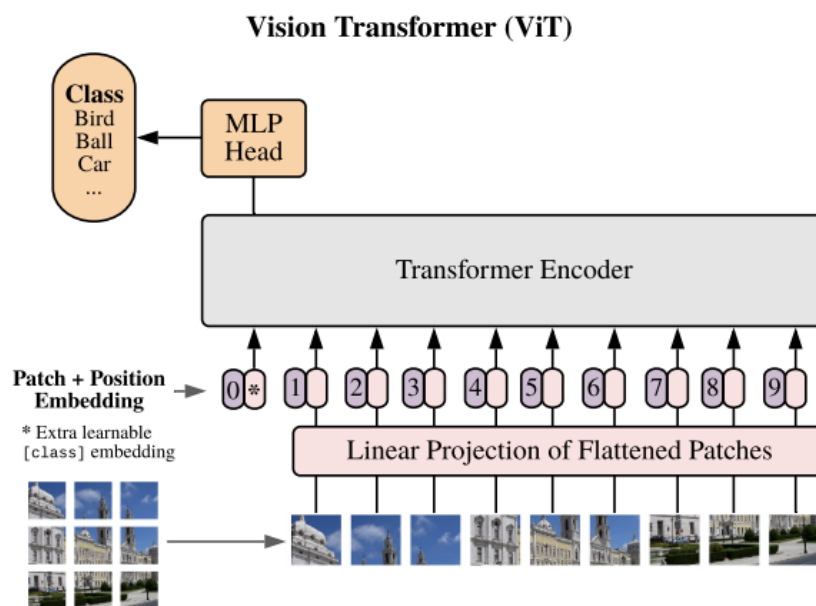


Figure 2.7: Architecture of the Vision Transformer [11]

There are many advantages of using a Vision Transformer (ViT) over a Convolutional Neural Network (CNN), including:

- **Handling Long-Range Relationships:** Vision Transformers excel in handling longrange relationships in images due to their self-attention mechanism. This capability allows them to capture global correlations between distant pixels/patches, which is often challenging for Convolutional Neural Networks (CNNs) [28].

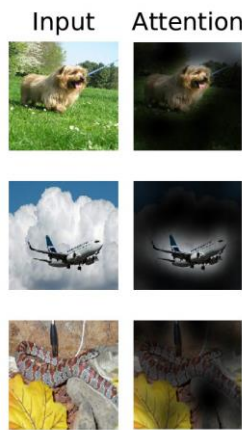


Figure 2.8: Attention maps [11]

Indeed, a CNN analyzes the input image by applying a set of kernels to a local region and sliding it across the entire image. This implies that in CNN architecture, the convolution operation is able to gather global information only in the higher layers, while in the lower layers, the convolution is computed across nearby pixels. In a ViT, self-attention allows each pixel/patch in the image to attend to all other pixels/patches simultaneously, regardless of their spatial proximity. This is a crucial advantage in computer vision tasks because it enables ViTs to understand how different parts of an image relate to each other in a holistic manner.

For instance, in an image of a beach scene, ViTs can capture the interplay between the distant horizon, the sky, the shoreline, and the people on the beach.

This holistic view enables ViTs to recognize patterns, context, and relationships that might be missed by traditional CNNs, which typically rely on local feature extraction through convolutional filters.

- **Scalability:** ViTs are designed to handle a wide range of image sizes. This adaptability is particularly beneficial because real-world images can vary significantly in dimensions. The ability of ViTs to handle images of varying sizes is derived from their architecture based on self-attention. In a ViT network, selfattention allows each element of the input (pixel or token) to interact with all other elements simultaneously. This means that there is no need to specify the exact dimensions of the image in advance, and the network will dynamically adapt to the input's dimensions. This flexibility is particularly valuable in scenarios where images may vary in terms of resolution or dimensions. For example, a ViT trained on low resolution images can be seamlessly used on high-resolution images without the need for significant reconfigurations. This greatly simplifies the development process and the practical application of ViTs in a wide range of computer vision contexts.
- **Greater Parallelism:** ViTs can perform attention in parallel across all image positions, which can lead to higher computational efficiency, especially on hardware accelerators, like GPUs and TPUs, leading to faster training and inference times. It enables ViTs to handle large-scale data and complex tasks with remarkable efficiency.

In the original paper [11], the architecture has been proven to generally outperform the CNN-based architecture ResNet (with a lower computational budget) when pre-trained on large datasets (see Appendix A for additional information about transfer learning). This is shown in figure 2.9, that compares some large ViT models pre-trained respectively on JFT300M (that comprises more than 300 millions of images, and isn’t publicly available) and ImageNet21k (14 millions of images) datasets to a large ResNet model. We can see that although the TPUv3-core-days spent for pre-training ViTs are significantly smaller with respect to the ResNet architecture, the latter is significantly outperformed by the Vision Transformers that are pre-trained on the JFT dataset.

	Ours-JFT (ViT-H/14)	Ours-JFT (ViT-L/16)	Ours-121k (ViT-L/16)	BiT-L (ResNet152x4)	Noisy Student (EfficientNet-L2)
ImageNet	88.55 $\pm$ 0.04	87.76 $\pm$ 0.03	85.30 $\pm$ 0.02	87.54 $\pm$ 0.02	88.4/88.5*
ImageNet ReaL	90.72 $\pm$ 0.05	90.54 $\pm$ 0.03	88.62 $\pm$ 0.05	90.54	90.55
CIFAR-10	99.50 $\pm$ 0.06	99.42 $\pm$ 0.03	99.15 $\pm$ 0.03	99.37 $\pm$ 0.06	—
CIFAR-100	94.55 $\pm$ 0.04	93.90 $\pm$ 0.05	93.25 $\pm$ 0.05	93.51 $\pm$ 0.08	—
Oxford-IIIT Pets	97.56 $\pm$ 0.03	97.32 $\pm$ 0.11	94.67 $\pm$ 0.15	96.62 $\pm$ 0.23	—
Oxford Flowers-102	99.68 $\pm$ 0.02	99.74 $\pm$ 0.00	99.61 $\pm$ 0.02	99.63 $\pm$ 0.03	—
VTAB (19 tasks)	77.63 $\pm$ 0.23	76.28 $\pm$ 0.46	72.72 $\pm$ 0.21	76.29 $\pm$ 1.70	—
TPUv3-core-days	2.5k	0.68k	0.23k	9.9k	12.3k

Figure 2.9: Transfer of large datasets on large models [11]

The ViT model trained on the smaller ImageNet21k dataset delivers results comparable to those of the CNN architecture, and it accomplishes this with only 1/40th of the computational power. In contrast, the following figure illustrates that when ViTs are pretrained on a smaller dataset (ImageNet with 1K classes, comprising 1.3 million images), they are unable to match the performance of ResNet models. Furthermore, the image demonstrates that with a larger pre-trained dataset, the size of the model becomes a crucial factor, and larger models achieve higher accuracy.

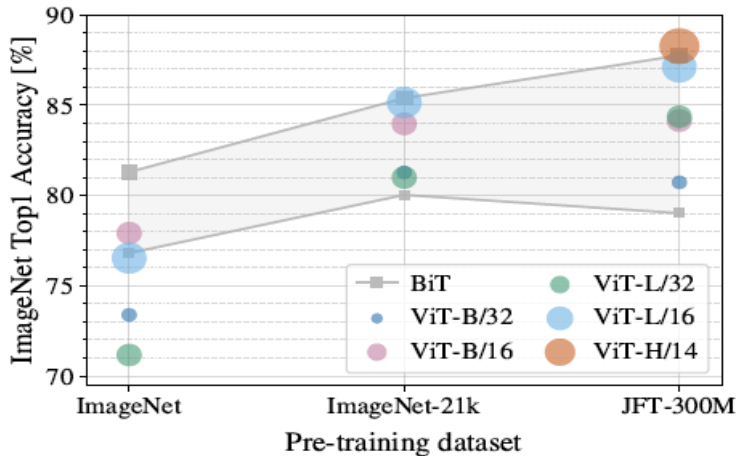


Figure 2.10: ViT and CNNs comparison with different pre-trained datasets sizes [11]

2.1.4. How to train a ViT: regularization and data augmentation

The study's findings indicate that ViT excels when trained on extensive datasets. However, a challenge arises in certain application domains where the available datasets are notably smaller, as is the case with this thesis. In 2021, Google Research Brain team [29] found that the combination of increased compute and techniques of Data Augmentation and Regularization can yield models with the same performance as models trained on an order of magnitude more training data. Furthermore, the study points out how transfer learning is always the best option when it comes to small datasets.

In particular, the paper shows that

- for models pre-trained with the mid-sized ImageNet-1k dataset, any kind of Data Augmentation or regularization improves the performances on all models
- for models pre-trained with ImageNet-21k dataset (which is 10x larger) there are two scenarios:
 - When training for a limited number of epochs, such as 30 epochs, any form of AugReg negatively impacts performance except for the largest models.
 - Expanding the training time to 300 epochs, AugReg benefits a broader range of models, although it still has a detrimental effect on the smaller ones.

2.2. Image segmentation

Image semantic segmentation is a fundamental task in the field of computer vision. It involves the identification, classification, and separation of specific regions within an image to recognize objects or structures of interest. This technique finds applications in a wide range of fields, including object recognition, medical diagnostics, autonomous driving, anomaly detection, and much more.

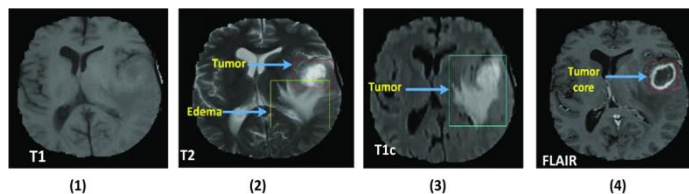


Figure 2.11: Example of image segmentation

In semantic segmentation, the process begins with a raw image and aims to divide it into different regions or segments, each representing a particular object or feature. This segmentation allows computer systems to better understand the image's content, identifying contours, shapes, and structures within it. This is particularly useful in scenarios where detailed information needs to be extracted from images, such as detecting tumors in a medical scan or isolating objects in a surveillance image.

Formally, we can define the goal of semantic segmentation task as the following:

Proposition 2.1. *Given an image and a finite set of classes $\Lambda = \{K_1, K_2, \dots, K_m\}$, associate to each pixel of the image a label from Λ .*

Several methods have been proposed to address this task, and one prominent approach is machine learning. In particular, we can train a model to label each pixel in a supervised fashion by supplying the algorithm with a set of input-output pairs:

- Input: the source image
- Output: the desired segmentation map

Throughout the training phase, the model attempts to generate segmentation maps and continuously refines itself by assessing its error using a specific loss function, ultimately achieving accurate predictions.

Chapter 3 - MRI Scans and Medical Image Segmentation

Magnetic Resonance Imaging (MRI) scans of the brain have revolutionized our understanding of the intricate structures and functions of the human brain. This medical imaging technique employs powerful magnetic fields and radio waves to produce detailed, high-resolution images of the brain's anatomy, allowing for the visualization of various brain regions, abnormalities, and pathologies. MRI scans are an essential tool in clinical diagnostics, neuroscience research, and the evaluation of neurological conditions, providing invaluable insights into brain health and facilitating the early detection and treatment of disorders affecting this vital organ. In section 3.1 we will introduce the basic concepts around MRI and brain scans, while in section 3.2 we will dive into the core of the thesis presenting the Brain Tumor Segmentation (often referred to as **BraTS**) challenge and its dataset.

3.1. Magnetic Resonance Imaging

3.1.1. MRI basics

Magnetic Resonance Imaging (MRI) is a non-invasive medical imaging technique that relies on the abundant presence of hydrogen atoms in the human body (water, fat, etc.). The fundamental principle of MRI is based on the magnetic properties of hydrogen nuclei, or protons. Within the body, these protons are typically randomly aligned. When a patient enters the MRI scanner, a strong external **magnetic field** is applied. This magnetic field causes the hydrogen protons to align themselves along its direction. This alignment sets the stage for image generation.

To create an image, a short burst of **radiofrequency (RF) energy** is directed at the patient's body. This RF pulse is precisely tuned to the resonant frequency of the hydrogen protons and points towards a different direction than the constant magnetic field [1, 8]. When the RF pulse is applied, it temporarily disrupts the alignment of the protons

obtained previously through the application of the magnetic field. As the RF pulse ends, the protons gradually return to their original aligned state, emitting RF signals in the process. Since water distribution is not homogeneous in the body, the variation in the detected electromagnetic field allow to determine the differences in tissue density and morphology across the entire body.

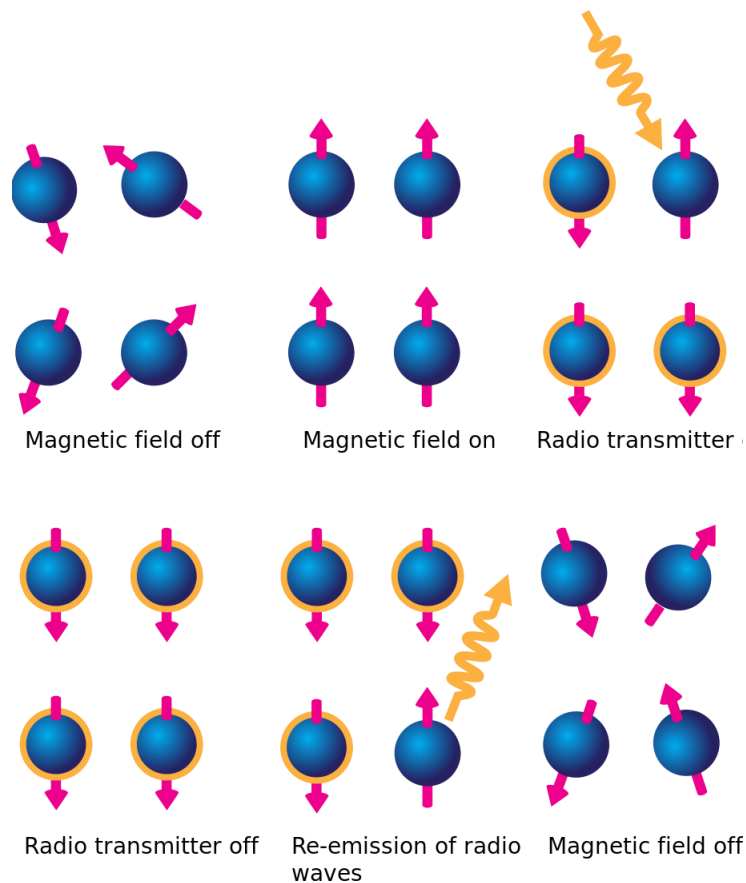


Figure 3.1: Effects of the application of magnetic field and burst of RF energy [1]

These emitted RF signals are picked up by receiver coils within the MRI machine. The signals contain information about the density and distribution of hydrogen protons within various tissues of the body.

To add spatial information to the images, gradient coils are utilized. These coils create varying magnetic field gradients across the body. By doing so, the MRI scanner can pinpoint the exact location of the emitted RF signals. This enables the creation of highly detailed 3D images, typically displayed as grayscale representations, where differences in brightness indicate variations in the density of hydrogen atoms in different tissues of the body.

Three-dimensional images are often represented slicing the scans through three planes:

- **Axial Plane:** a horizontal plane that divides the body or object into upper and lower parts.
- **Sagittal Plane:** a vertical plane that divides the body into left and right halves.
- **Coronal Plane:** a vertical plane that divides the body into front and back sections.

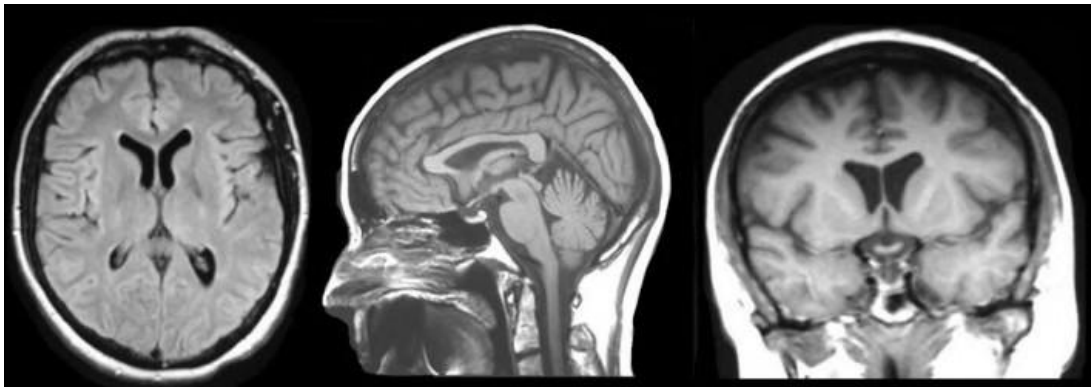


Figure 3.2: Axial, sagittal and coronal planes of a brain MRI scan [10]

Different types of images can be produced by manipulating the sequence of applied and captured RF pulses. This is done through two main parameters [10]:

- **Repetition Time (TR):** refers to the duration between consecutive pulse sequences administered to the same slice.
- **Time to Echo (TE):** indicates the time between the delivery of the RF pulse and the receipt of the echo signal.

In particular, different tissues can be characterized by two distinct relaxation times T_1 and T_2 [10].

- **T_1 (longitudinal relaxation time):** the time constant governing the speed at which excited protons revert to their equilibrium state. It signifies the duration it takes for spinning protons to reestablish alignment with the external magnetic field.
- **T_2 (transverse relaxation time):** the time constant that dictates how quickly excited protons attain equilibrium or lose synchronicity with each other. T_2 represents the duration it takes for spinning protons to lose phase coherence among nuclei spinning at right angles to the primary magnetic field.

3.1.2. MRI modalities

Based on this premise, various types of images can be produced by modulating the TR and TE times and measuring T1 and T2 values. In particular, we mention:

- **T1-weighted images:** during a T1 sequence, protons aligned along the main magnetic field gradually return to their equilibrium position. This sequence is ideal for visualizing the anatomical structures of the body since tissues with longer T1 relaxation times appear brighter, while those with shorter T1 relaxation times appear darker [27]. For instance, in T1 brain images, brain tissue appears brighter than cerebrospinal fluid (CSF), allowing for a clear visualization of cerebral structures. These images are produced using short TR and TE times.
- **T1ce (T1 with Contrast Enhancement):** The T1ce sequence is a variant of the standard T1 sequence used in combination with a **contrast agent**, such as **gadolinium**. This contrast agent is administered to the patient and accumulates in areas of interest, such as lesions, tumors, or inflammations, mostly because of leaky blood vessels. During the acquisition of T1ce images, areas where the contrast agent is concentrated appear very bright, while the rest of the tissues maintain their standard T1 appearance (comparison in figure 3.3). This sequence is crucial for highlighting pathologies and lesions that might otherwise go undiagnosed.

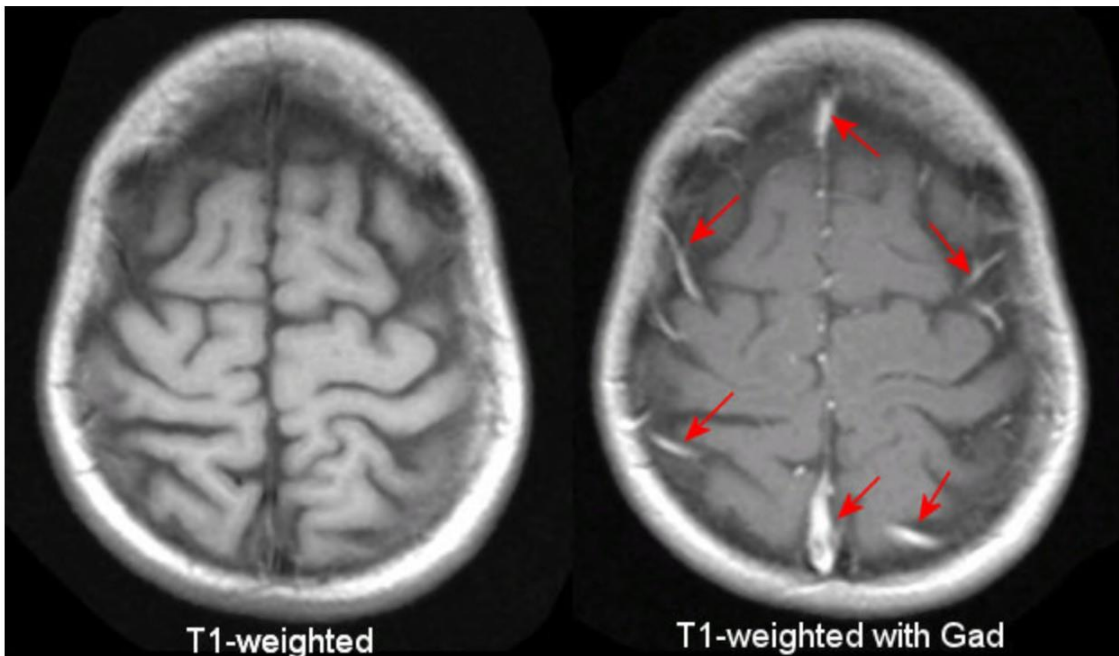


Figure 3.3: Differences in brightness of blood vessels between T1 and T1ce scans [10]

- **T2-weighted images:** these kind of images, produced using longer TR and TE times, represent the difference in the T2 (transverse relaxation) time across the scanned area. In these images, tissues with longer T2 relaxation times appear brighter, while those with shorter T2 relaxation times appear darker. The T2 sequence is widely used to detect lesions, inflammations, edema, and other pathological conditions in soft tissues. This is due to the fact that pathological processes often lead to an accumulation of fluids within the tissues. Since fluids tend to have longer T2 relaxation times, in these images brain lesions or edema appear brighter than the surrounding brain tissue, aiding in diagnosis and assessment [25]. With respect to T1-weighted images, cerebrospinal fluid appears brighter.
- **FLAIR (Fluid Attenuated Inversion Recovery):** Similar to T2 sequence, except for the longer TR and TE times [10], the FLAIR sequence is designed to suppress the signal from fluids of the body, in particular cerebrospinal fluid (CSF) within the brain [24]. This suppression enhances the visualization of lesions or pathologies in the brain, as CSF can often obscure critical details. In these images, CSF appears very dark, while the surrounding tissues appear brighter, making any anomalies more apparent. The FLAIR sequence is commonly used in the diagnosis of neurological diseases, including brain tumors.

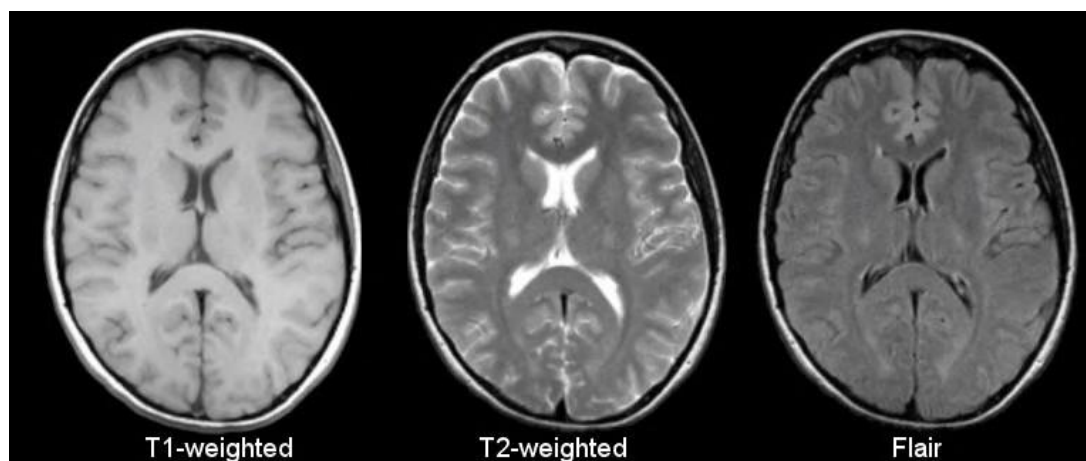


Figure 3.4: T1, T2 and FLAIR axial plan comparison [10]

3.1.3. Brain Tumors

A brain tumor is an abnormal growth of cells within the brain. These growths can be benign (non-cancerous) or malignant (cancerous) and can originate from brain tissue or spread from other parts of the body. Brain tumors can vary in size, location, and behavior. The most prevalent type of brain tumor in adults is the **glioma**, accounting for almost 80 percent of malignant brain tumors. They arise from the supporting cells of the brain, called the glia. In particular, **Glioblastoma multiforme (GBM)** represents the most invasive form of glial tumors. These tumors exhibit rapid growth, a propensity to spread to surrounding tissues, and a generally unfavorable prognosis. They can consist of various cell types, including astrocytes and oligodendrocytes. GBM primarily affects individuals between the ages of 50 and 70 and shows a higher incidence in men compared to women [26].

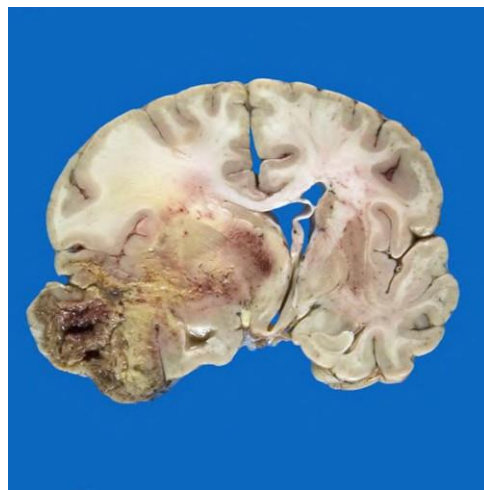


Figure 3.5: Glioblastoma multiforme [12]

Brain tumors are believed to originate from the damage to specific genes on a cell's chromosomes, leading to their malfunction. These genes are in charge to control the rate of cell division, repair defects in other genes, and trigger self-destruction if the damage is irreparable. If they are damaged and can't function normally, once a cell starts dividing rapidly it can progress into a tumor. The body's immune system can serve as an initial defense, ideally detecting and eliminating abnormal cells. Tumors may produce substances that obstruct the immune system's recognition of these abnormal cells and then grow uncontrollably [26].

3.2. Brain Tumor Segmentation challenge

Brain tumor segmentation is a critical task in medical image analysis that involves the identification and delineation of various tumor components within brain MRI scans. Accurate segmentation is essential for diagnosis, treatment planning, and monitoring the progression of brain tumors. One of the well-known challenges associated with brain tumor segmentation is the **Brain Tumor Segmentation Challenge (BraTS)**.

The BraTS challenge [4–7, 20] is an annual competition in the field of medical image analysis. It is organized by the Medical Image Computing and Computer Assisted Interventions (MICCAI) community and focuses on the segmentation of brain tumors using multi-modal MRI scans. The challenge aims to promote research and the development of advanced algorithms for brain tumor segmentation, as well as to benchmark and compare the performance of different methods.

Participants are provided with MRI scans that include various modalities, such as

- T1-weighted modality
- T2-weighted modality
- fluid-attenuated inversion recovery (FLAIR) modality
- post-contrast T1-weighted (T1ce or, equivalently, T1Gd) modality

These different modalities offer complementary information for accurate tumor segmentation.

The challenge addresses the segmentation of different tumor types, including

- **Necrotic Tumor Core (NCR - label 1):** This label corresponds to the core of the tumor, where the tissue is necrotic, meaning the cells in this region have died. Necrosis is a common feature in aggressive brain tumors and is often associated with tissue death and lack of blood supply. Accurate segmentation of this core is crucial for understanding the tumor's inner structure.
- **Edematous/Invaded Tissue (ED - label 2):** This label represents the region surrounding the tumor where the brain tissue has been affected by edema. Edema is characterized by the accumulation of fluid, which can lead to swelling and changes in tissue density. In the context of brain tumors, the edematous region typically indicates areas where the tumor has invaded and impacted surrounding healthy brain tissue. Accurate segmentation of this region is vital for assessing the tumor's impact on the surrounding brain and planning surgical or therapeutic interventions.

- **GD-Enhancing Tumor (ET - label 4):** This label refers to the region of the tumor that exhibits enhancement after the administration of a contrast agent during MRI imaging. The GD-enhancing tumor core typically represents the most actively growing and vascularized part of the tumor. This region is of particular interest in treatment planning and assessment because it often corresponds to the most aggressive and actively growing tumor cells. Accurate segmentation of this area is essential for planning surgical resections and evaluating the response to therapies.

The BraTS challenge uses specific evaluation metrics to assess the accuracy of segmentation results, including the Dice Similarity Coefficient (DSC), on three different sub-regions [3]:

- **Enhancing tumor (ET):** described previously, is the area defined by regions exhibiting hyper-intensity in T1ce when compared to T1, as well as when contrasted with "healthy" white matter in T1ce.
- **Tumor core (TC):** defines the central mass of the tumor, typically the area targeted for surgical removal. It encompasses both the necrotic part of the tumor and the enhancing tumor.
- **Whole tumor (WT):** the full extent of the disease. It entails the tumor core and the peritumoral edematous region.

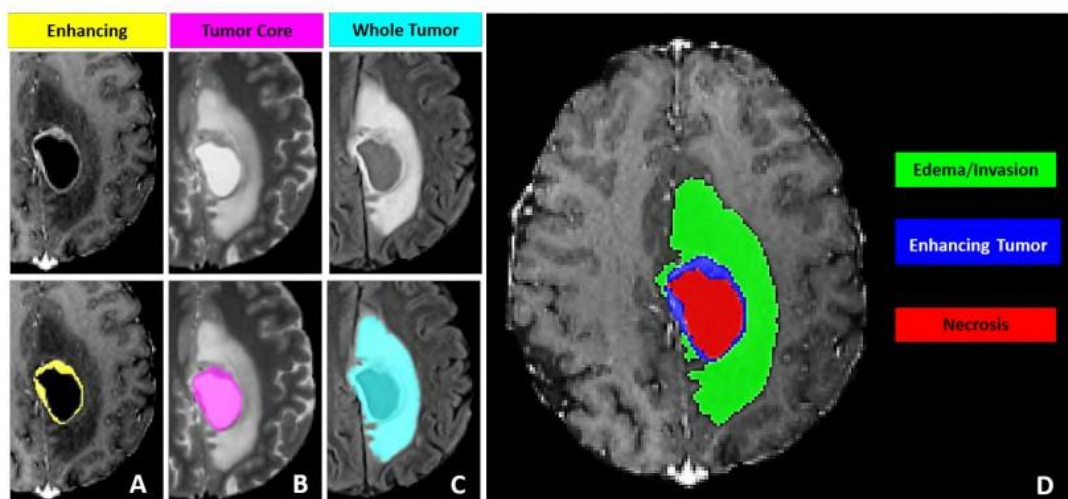


Figure 3.6: Representation [3] of the three sub-regions (A, B, C) considered for the evaluation and the three labels (D)

The challenge has significantly advanced the field of brain tumor segmentation by serving as a benchmark for state-of-the-art methods. It has encouraged the development of

deep learning models, including convolutional neural networks (CNNs), that have shown impressive results in recent editions. The developments and innovations emerging from the BraTS challenge have the potential to improve clinical practice by providing more accurate and reproducible tumor segmentations. This can lead to better patient outcomes, treatment planning, and assessment.

In 2020, Andriy Myronenko and Ali Hatamizadeh from NVIDIA proposed an architecture [23] based on Convolutional Neural Networks for achieving the brain tumor segmentation task using the BraTS2019 challenge dataset, and other teams from around the world proposed alternative approaches [2, 15, 16, 32, 33] to address the same problem. In the following chapter we will expose the approaches that we have adopted to face this challenge, and in Chapter 5 we will compare the results with these studies.

Chapter 4 -Transformer-based BrainTumor Segmentation

4.1. 3DMAT

In this experiment, we employ the **3D Medical Axial Transformer (MAT)**, a lightweight Transformer-based architecture designed for efficient **3D brain tumor segmentation** on multi-modal MRI scans from the BraTS challenge. Below, we outline the model's design and its advantages over existing approaches.

4.2. 3DMAT architecture

The 3DMAT architecture is composed by an **encoder**, that analyze the input at different resolutions and using a shifting window mechanism, and a CNN-based decoder that reconstructs the three-dimensional segmentation map of the input scan.

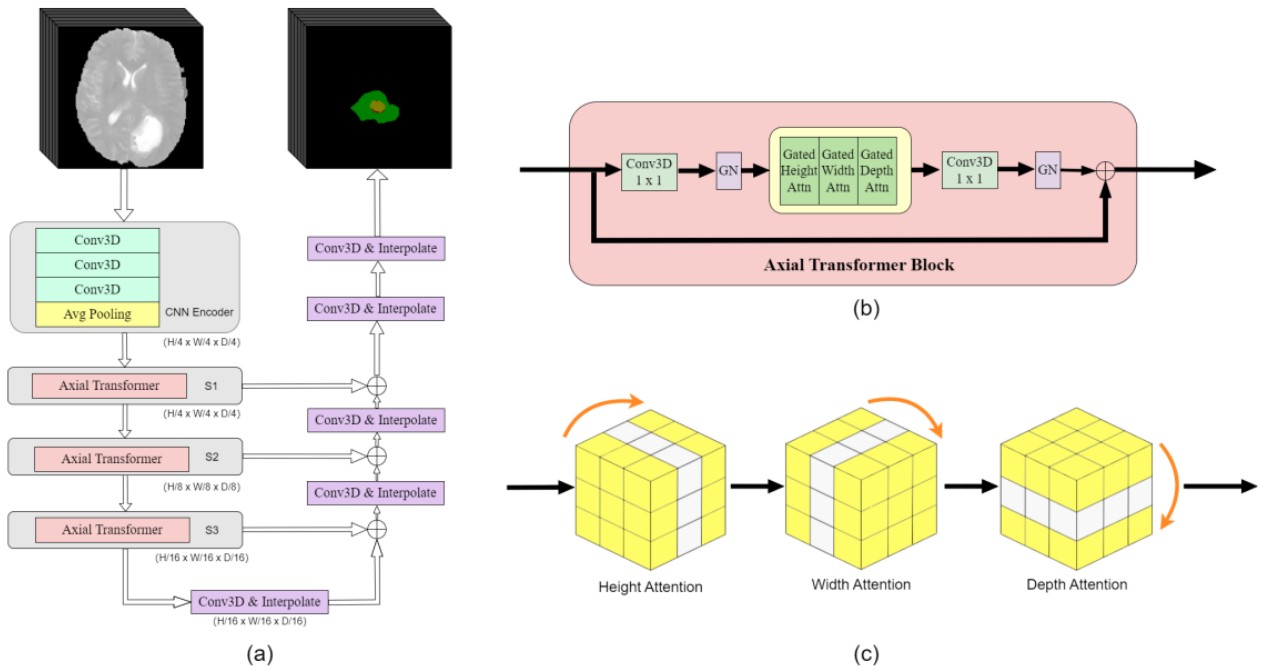


Figure 4.7: 3DMAT architecture [13]

(a) Architecture overview of MAT (b) designs of Axial Transformer Block (c) Schematic diagram of the Axial Attention calculation method

Encoder

The MAT encoder employs a hybrid design combining convolutional layers with axial Transformer blocks to efficiently process 3D medical images. After initial feature extraction via three CNN blocks (7×7×7 and 5×5×5 convolutions with GroupNorm), the model applies gated axial attention in three stages (S1-S3). This mechanism decomposes

computationally expensive 3D self-attention into three sequential 1D attentions along the depth, height, and width axes. Each axial attention incorporates learnable gating parameters to dynamically weight positional encodings, preventing noise amplification in early training phases. The multi-stage design progressively doubles channels while halving spatial dimensions through average pooling, yielding hierarchical 3D features. Residual connections and GroupNorm stabilize training, enabling effective learning of long-range dependencies with only 11.7M parameters.

Key Formula - Gated Axial Attention (Height Axis):

For a query position (i,j,k) , the output(y_{height}) is computed as:

$$y_{\text{height}_{ijk}} = \sum_{h=1}^H \text{Softmax} \left(q_{ijk}^T k_{ihk} + q_{ijk}^T r_{ihk}^q + k_{ihk}^T r_{ihk}^k \right) (v_{ihk} + r_{ihk}^v)$$

where:

- q, k, v : Query, key, value vectors
- r^q, r^k, r^v : Relative positional encodings
- G_q, G_k, G_v : Learnable gates (scalars) controlling positional term contributions
- H : Height dimension size

The depth/width axes follow analogous formulations (Eq. 7-8 in paper). This decomposition reduces FLOPs from $O(DHWDHW)$ to $O(DHW(D+H+W))$ while preserving cross-slice context.

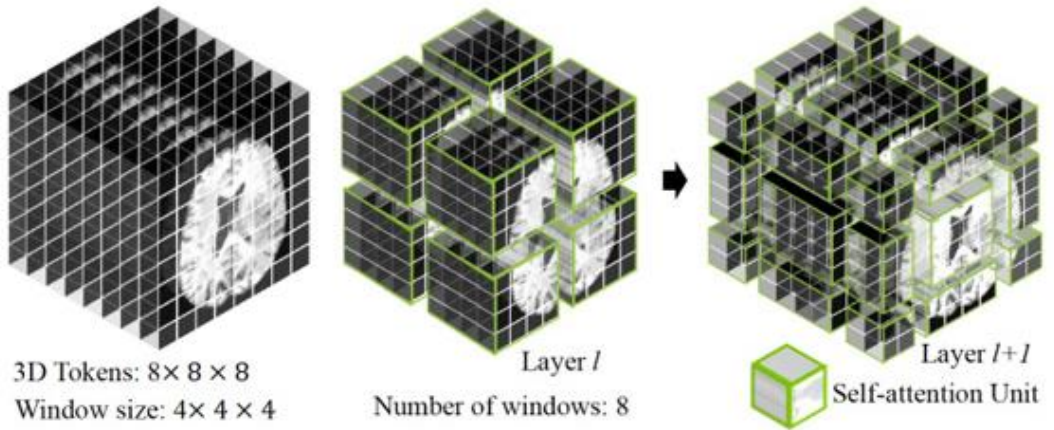


Figure 4.8: Shifted-window self-attention mechanism [13]

Decoder

The decoder in the 3D Medical Axial Transformer (MAT) is designed to reconstruct high-resolution segmentation maps from the deep semantic features produced by the axial Transformer encoder. It adopts a CNN-based Unet-like decoder architecture, which is symmetrical to the encoder. The decoder consists of five CNN-interpolation blocks, where the first block has a stride of 1 and the remaining four use stride 2 to progressively upsample the feature maps, effectively reversing the downsampling done in the encoder. A ReLU activation is applied after each convolution. The upsampling process is accompanied by skip connections from the encoder stages, allowing the decoder to recover spatial details lost during encoding. However, unlike traditional Unet which uses four skip connections, MAT uses only three to avoid introducing noise from shallow layers, as supported by previous studies.

$$\begin{aligned}
X_1 &= \text{ConvInterp}(X_{\text{encoder}}) \in \mathbb{R}^{C \times \frac{D}{16} \times \frac{H}{16} \times \frac{W}{16}} \\
X_2 &= \text{ConvInterp}(X_1) \in \mathbb{R}^{C \times \frac{D}{8} \times \frac{H}{8} \times \frac{W}{8}} \\
X_3 &= \text{ConvInterp}(X_2 + \text{Skip}_3) \in \mathbb{R}^{C \times \frac{D}{4} \times \frac{H}{4} \times \frac{W}{4}} \\
X_4 &= \text{ConvInterp}(X_3 + \text{Skip}_2) \in \mathbb{R}^{C \times \frac{D}{2} \times \frac{H}{2} \times \frac{W}{2}} \\
X_5 &= \text{ConvInterp}(X_4 + \text{Skip}_1) \in \mathbb{R}^{C \times D \times H \times W}
\end{aligned}$$

Loss function

For determining the training error the paper proposes the **voxel-wise soft DICE Loss**

$$\text{Loss} = 1 - \frac{2}{J} \sum_{j=1}^J \frac{\sum_{i=1}^I G_{i,j} Y_{i,j}}{\sum_{i=1}^I G_{i,j}^2 + \sum_{i=1}^I Y_{i,j}^2} \quad (4.2)$$

where:

- I and J denote respectively the number of voxels and the number of classes
- $Y_{i,j}$ represents the probability predicted by the model that the voxel i belongs to class j

4.2.1. Data preparation

For this experiment, we utilized the same dataset employed for the Segmenter Models, organized into multiple volumes rather than slices. The training set comprises 227 patients, the validation set includes 49 patients, and the test set is comprised of 59 patients. Each patient's data is collected and processed into a set of four NIFTI files, corresponding to different modalities: FLAIR, T1, T1 with contrast (T1ce), and T2. Additionally, for each patient, there is a ground-truth NIFTI file containing the segmentation maps for the classes defined in the BraTS dataset. Each MRI scan has a dimension of $180 \times 180 \times 128$ voxels.

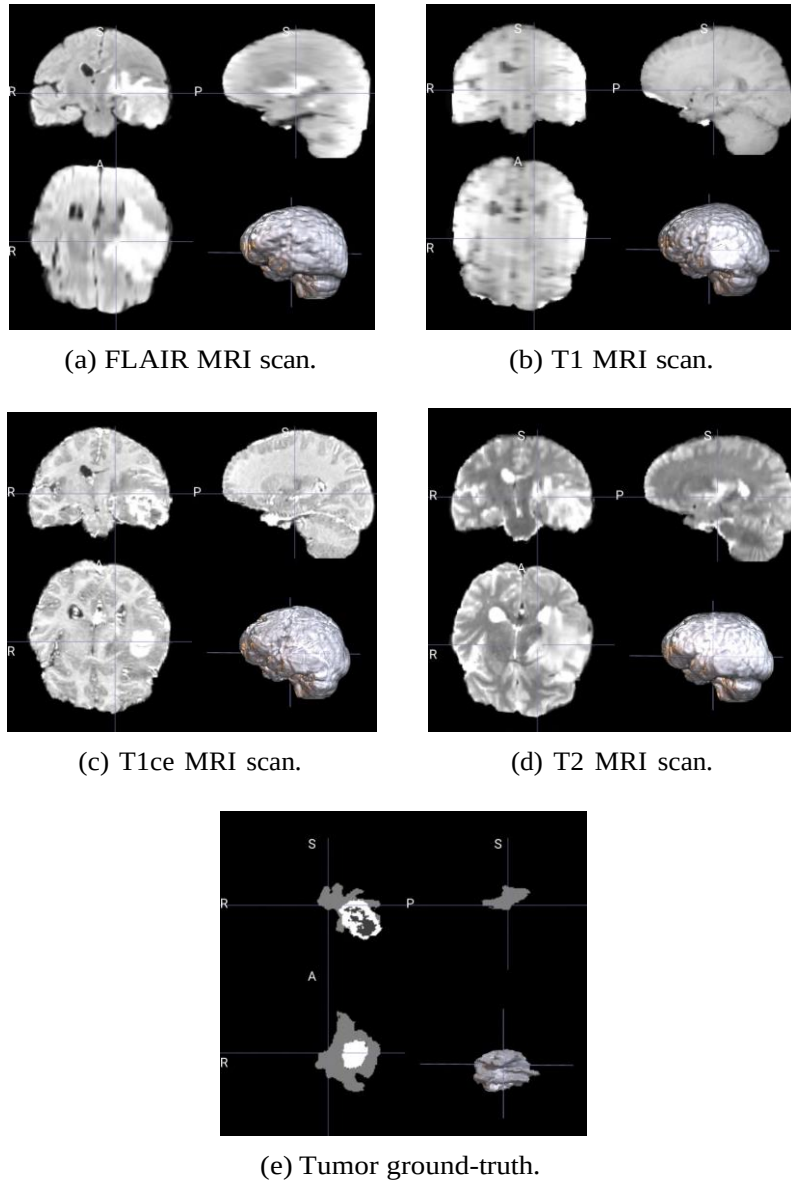


Figure 4.9: An example of MRI scan and its respective tumor ground-truth

Preprocessing and data augmentation

To enhance the model's performance, we employ various preprocessing and data augmentation techniques on the input scans. These techniques are described in detail in the official documentation of MONAI [22] and encompass, in this order:

- **Foreground crop:** cropping the original scan using a bounding box that excludes most of the foreground, which is not interesting for the analysis. This function is commonly utilized to assist in training and evaluation when dealing with medical images where the relevant or valid part occupies a relatively small portion of the entire image.
- **Random Spatial Cropping:** crop the image with a randomly sized or a specific-sized Region of Interest (ROI).
- **Random Flipping:** the images are subject to mirroring, which occurs independently across all three spatial dimensions with a 50 percent probability.
- **Random intensity scaling and shifting:** each of the input scans undergoes a random intensity scaling and shifting operation.

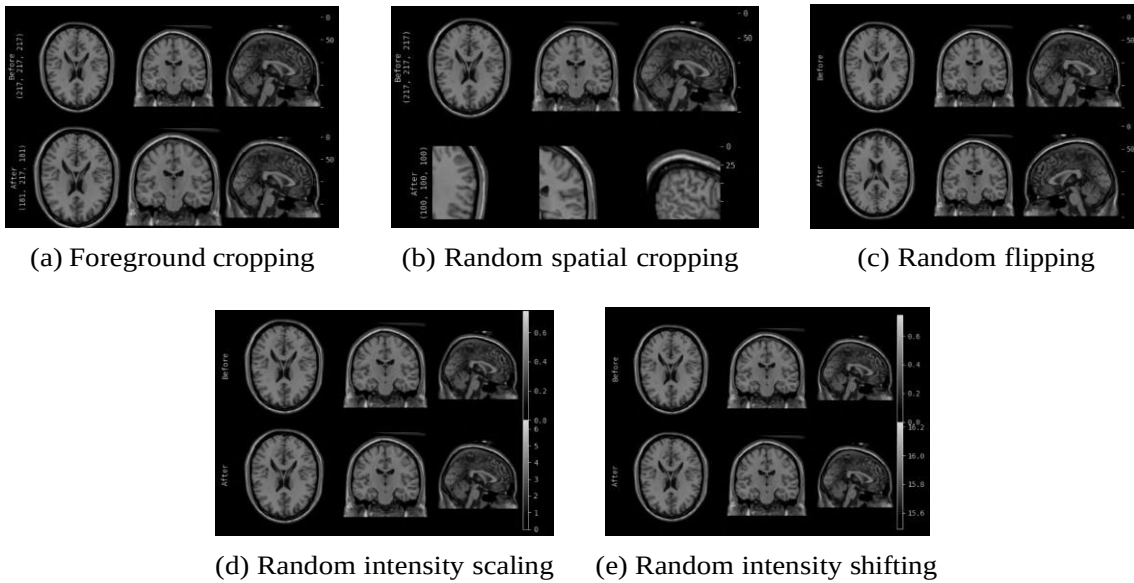


Figure 4.12: The preprocessing techniques applied to the 3DMAT dataset

4.2 Evaluation metrics

In the realm of machine learning, particularly in tasks like semantic segmentation, evaluation metrics are indispensable tools for quantifying the performance of models and algorithms. Central to these metrics are four fundamental quantities:

- **True Positives (TP):** true positives in semantic segmentation refer to those pixels or regions that the model correctly identifies as part of a specific class or object. Essentially, TP indicates the model's ability to accurately capture and segment the regions of interest within an image or scene.
- **True Negatives (TN):** true negatives, in this context, represent the pixels or regions that the model correctly recognizes as not belonging to a specific class or object. TN showcases the model's accuracy in excluding regions that do not pertain to the target class, ensuring precision in segmentation.
- **False Positives (FP):** false positives occur when the model incorrectly identifies pixels or regions as part of a class when they do not belong to that class. In semantic segmentation, FP may lead to the model's misinterpretation of background elements as foreground objects, potentially resulting in over-segmentation.
- **False Negatives (FN):** false negatives, in the context of semantic segmentation, are those pixels or regions that the model fails to identify as part of a specific class or object when they should have been. FN can lead to the omission of relevant objects or regions, causing under-segmentation and incomplete analysis.

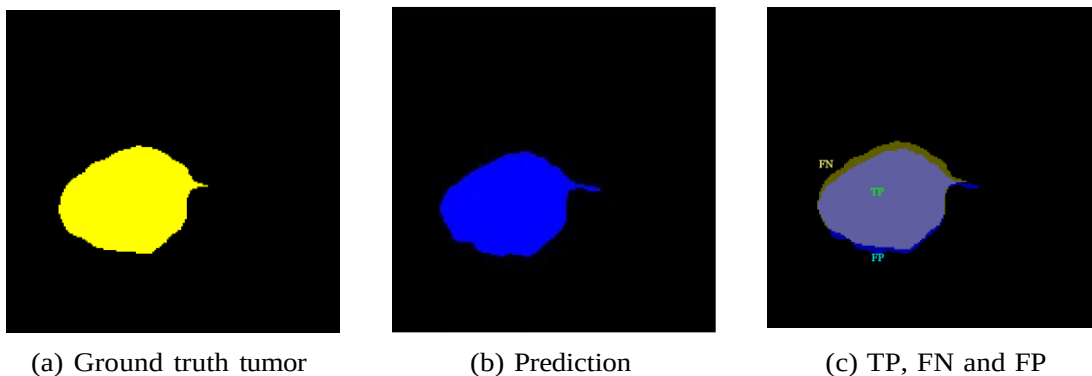


Figure 4.13: Visual example of TP, FP and FN for the Whole Tumor class

In semantic segmentation tasks, these concepts are crucial for assessing the model's performance. We will now delve into how these concepts are employed to calculate two commonly used metrics in semantic segmentation tasks.

4.2.1. DICE score

The DICE score, also known as the "Sørensen–Dice coefficient", is a metric used to evaluate the accuracy of segmentation in various image processing tasks, including semantic segmentation. It quantifies the overlap between the predicted segmentation (by a model) and the ground truth (the actual segmentation in the dataset). The DICE score is calculated as the intersection of the predicted and ground truth regions divided by the average size of the two regions.

$$\text{DICE} = \frac{2 \times |\text{PREDICTION} \cap \text{GROUND-TRUTH}|}{|\text{PREDICTION}| + |\text{GROUND-TRUTH}|} = \frac{2 \times \text{TP}}{2 \times \text{TP} + \text{FP} + \text{FN}} \quad (4.3)$$

It provides a value between 0 and 1, where 1 indicates a perfect match between the prediction and the ground truth, while lower values represent less accurate segmentations. In essence, it measures the similarity or agreement between the model's output and the expected results, making it a valuable tool for evaluating segmentation performance in machine learning and computer vision applications.

4.2.2. Intersection over Union score

Intersection over Union (IoU), also known as the Jaccard Index, is another crucial metric in semantic segmentation tasks. It assesses the quality of the segmentation by measuring the overlap between the predicted and ground truth regions. IoU is computed as the ratio of the intersection of the two regions to their union.

$$\text{IoU} = \frac{|\text{PREDICTION} \cap \text{GROUND-TRUTH}|}{|\text{PREDICTION} \cup \text{GROUND-TRUTH}|} = \frac{\text{TP}}{\text{TP} + \text{FP} + \text{FN}} \quad (4.4)$$

The value of IoU ranges from 0 to 1, where 1 indicates a perfect overlap between the prediction and the ground truth, while lower values suggest less accurate segmentation. In simpler terms, it evaluates how well the predicted segmentation aligns with the actual segmentation, making it a valuable measure for assessing the performance of machine learning models in semantic segmentation tasks. Higher IoU scores indicate more precise and accurate segmentations, while lower scores indicate less accurate results.

Chapter 5-Experimental design and results

In this chapter, we will present and analyze the design and the results of our work. In section 5.1 we will list the models that we have trained with the two different architectures and provide details about the experimental setup. Then, in section 5.2, we will delve into the quantitative outcomes of the models we trained for both architectures. Following that, in section 5.3, we will showcase several visual examples. Lastly, in section 5.4, we will examine specific edge cases where the model predictions may not be accurate.

5.1. Experimental design

5.1.1. 3DMAT Model

We trained three variants of the **MAT** model, optimizing for memory efficiency and segmentation performance on 3D brain MRI scans. Due to GPU constraints, all models used a **batch size of 1**, leveraging MAT’s lightweight design to fit within 16GB memory. The architecture followed the original paper’s specifications with the following key hyperparameters:

- **Patch Size:** No patch partitioning (MAT processes full 3D volumes directly).
- **Axial Attention Blocks:** 7 blocks total (stages S1:1, S2:2, S3:4).
- **Attention Mechanism:**
 - Gated axial attention along depth, height, and width axes.
 - Window-free computation (unlike 3DMAT’s $7\times 7\times 7$ local windows).
- **Learning Rate:** 10^{-4} to 10^{-5} with AdamW optimizer, warm-up (10 epochs), and cosine decay.

Model Variants

The three trained models differed in:

1. Embedding Dimension (CnnCnn):

- Varied base channels (64, 96, 128) to adjust feature capacity.

2. Input ROI Size:

- BraTS2018: $160\times 224\times 224$ (full tumor context).
- BraTS2021: $128\times 160\times 160$ (memory-optimized).

3. Augmentation Intensity:

- Random flips/rotations (50% probability) scaled by ROI size.

The trained models differ in both the dimension of the embedding space and the size of the region of interest used for pre-processing/data augmentation techniques.

The classes taken into account for the soft DICE loss correspond to the subregions defined in the context of the BraTS 2019 challenge, specifically, the Whole Tumor, Tumor Core, and Enhancing Tumor.

5.1.2. Training details

Before presenting the results, there are some considerations to do about the training phase:

During training, all MAT models underwent 2500 epochs of optimization on NVIDIA Tesla T4 and A100 GPUs, with their lightweight architecture (11.7M parameters) enabling efficient training even with a batch size of 1. Analysis of training logs revealed no significant overfitting, attributed to the model's novel self-distillation mechanism (DLB) with warm-up scheduling and gated axial attention design. Unlike traditional approaches, MAT employed a combined loss function (40% cross-entropy, 60% Dice) with dynamic DLB weighting (α), eliminating the need for separate necrosis/edema evaluation. Final test scores were computed using the last epoch's checkpoint, as the model demonstrated stable convergence throughout training. This robust performance stems from MAT's end-to-end 3D processing and axial attention mechanism, which maintained memory efficiency (16GB GPU sufficient) while capturing full volumetric context - a significant advantage over patch-based methods requiring higher computational resources.

5.1.3. Quantitative results

5.1.4. 3DMAT

Now, we present the test scores of the 3DMAT models. In contrast to the Segmenter models, all these models experienced overfitting before the completion of training. Therefore, in this subsection and the next one, we present the test results based on the best checkpoint available.

3DMAT models - Whole Tumor DICE

Model	Mean $\pm$ Std	Median	Q1	Q3	Min	Max
MAT-64 (Base)	93.05$\pm$1.49	93.41	92.12	94.33	88.76	95.62
MAT-96	93.21$\pm$0.93	93.55	92.87	94.01	90.14	95.88

Table 5.1: Whole Tumor DICE performance for 3DMAT models on the test set

3DMAT models - Tumor Core DICE

Model	Mean $\pm$ Std	Median	Q1	Q3	Min	Max
MAT-64 (Base)	87.91$\pm$2.47	88.63	86.29	89.94	80.05	92.77
MAT-96	91.91$\pm$1.92	92.16	90.88	93.04	85.22	94.50

Table 5.2: Tumor Core DICE performance for 3DMAT models on the test set

3DMAT models - Enhancing Tumor DICE

Model	Mean $\pm$ Std	Median	Q1	Q3	Min	Max
MAT-64 (Base)	82.81$\pm$2.41	83.52	81.67	84.90	75.33	87.44
MAT-96	85.05$\pm$1.98	85.61	84.12	86.33	78.91	88.02

Table 5.3: Enhancing Tumor DICE performance for 3DMAT models on the test set

3DMAT models - Summary

Model	Whole Tumor	Tumor Core	Enhancing Tumor
MAT-64	93.05 $\pm$ 1.49	87.91 $\pm$ 2.47	82.81 $\pm$ 2.41
MAT-96	93.21 $\pm$ 0.93	91.91 $\pm$ 1.92	85.05 $\pm$ 1.98

Table 5.4: Summary of 3DMAT model performance on the test set

From these scores, we can infer that training the model with a larger region of interest (ROI) may contribute to better generalization to unseen data. Specifically, the model *SWIN64-48*, trained using an ROI of $64 \times 64 \times 64$ voxels, yields lower results in all subregions compared to the other two models, which employ an ROI of $96 \times 96 \times 96$. This effect can be attributed to the inherent nature of Transformers, which leverage information from the entire volume under consideration, enabling more precise predictions when considering larger areas. Simultaneously, we observe that expanding the embedding space does not notably enhance performance for the Whole Tumor subregion. Instead, it slightly worsens the score for the Tumor Core, resulting in a higher median but lower mean and higher variance. However, this expansion proves beneficial in enhancing generalization for the Enhancing Tumor subregion. Now, let's compare our obtained results with the scores reported in the 3DMAT paper [13]:

3DMAT score - original paper

Metric	MAT (Paper)	MAT-96 (Ours)
WT Dice	93.05	93.21
TC Dice	87.91	91.91
ET Dice	82.81	85.05

Table 5.5: DICE performance on test set reported in 3DMAT paper [13]

Our models did not achieve the same level of performance as the model presented in the paper. The results differ slightly for what concerns the WT and TC subregions, but drop sensibly for the Enhancing Tumor subregion.

Considering that the paper’s model is trained on the Brats2021 dataset, which includes 1251 cases for training and 219 patients for validation, our models, trained on less than a fifth of the patients, still demonstrate competitive results.

Moreover, our analysis involved a subset of the original test set. In this subset, patients with exceptionally small enhancing tumor regions may introduce outliers in the computation of the DICE metric. The DICE metric, particularly sensitive to classes that present few voxels, can significantly impact the metric even with a minimal error in voxel prediction, thereby influencing a lower overall mean. This phenomenon is attributed to the inherent mathematical characteristics of the DICE score. In instances where a specific subregion has minimal extent, even slight errors can disproportionately impact the score due to their significant influence on the ratio between true positives, false negatives, and false positives.

In particular, when the region is not present in the ground truth, two scenarios may arise:

- If there is at least one pixel/voxel of the class incorrectly predicted, the DICE formula will yield 0 (true positives) in the numerator, resulting in a null DICE score, even with just one false positive.
- if the model accurately predicts the absence of that class, we will have True Positives, False Positives, and False Negatives all set to 0 simultaneously. This condition causes the DICE formula to have 0 in both the numerator and denominator, resulting in a not-a-number value.

This suggests that regardless of the model’s overall ability to predict that subregion, the

metric will consistently be 0 or undefined, significantly influencing the average score and making it challenging to interpret. Consequently, the computation of the DICE score in such cases is entirely meaningless.

To address this issue, we opted to exclude from the test set the patients for whom a subregion is not present in the ground truth. Specifically, we removed the 46th and 56th patients from the test set, which ground truth has no voxels associated to the Enhancing Tumor class, reducing the number of test patients from 59 to 57.

Considering the factors we’ve discussed, and given the uncertainty surrounding the class composition in the ground truth of each Brats2021 test patient, it’s crucial to perceive the comparison between our scores and the one reported in the original paper as indicative rather than conclusive.

5.1.5. Transformers vs CNNs: result comparison

In this subsection, we’ll compare the results of our experiments with various state-of-the-art architectures. Our focus is on comparing the Transformer-based models’ outcomes with those from related works that adopt diverse strategies within the context of the BraTS 2019 challenge. In particular, while some of these models rely exclusively on CNNs, others employ a hybrid approach.

For this comparison, we will consider the following studies:

- The paper from Andriy Myronenko and Ali Hatamizadeh [23], which introduces a method employing a typical U-Net encoder-decoder structure for analyzing three dimensional scans.
- The study conducted by Xiangyu Li, Gongning Luo, and Kuanquan Wang [16], which presents an innovative approach using multi-step cascaded networks in sequence. The initial network reconstructs the Whole Tumor segmentation map of a random cropped sub-volume of the input scan, utilizing it as a filter for the input T1ce image. Subsequently, the second segmentation network takes as input the filtered image, which comprises only tumoral tissue, and focuses on the sub-nested Tumor Core region, which, in turn, guides the final segmentation network to identify the Enhancing Tumor subregion. Each segmentation network consists in a classical UNet based on CNNs.
- The work of Rupal R. Agravat and Mehul S. Raval[2], which proposes a fully convolutional neural network designed for segmenting 2D slices.
- The 3D U-Net architecture [32] proposed by Feifan Wang et Al.
- The TransBTS[33], an approach that adopts a Transformer architecture as a bottleneck of a U-Net encoder-decoder structure for the analysis of 3D MRI scans.

- The SegTran [15], which uses a CNN to extract features of MRI scans and feeds them to a series of Transformer layers that implement a peculiar technique called Squeezeand-Expansion.

Brats2019 DICE results comparison

	Whole Tumor (WT)	Tumor Core (TC)	Enhanc ing Tumor (ET)
cons-t1-256-4-4-2	67.47	43.11	16.85
cons-t1-256-8-8-2	69.21	42.63	17.96
cons-flair-256-8-8-2	81.26	46.23	22.89
mod256-8-8-2	84.75	75.08	62.28
mod256-8-8-4	84.81	74.61	62.70
mod256-16-16-8	83.11	72.06	61.18
pret192-12-3-8	86.56	80.14	64.47
pret768-12-12-8	83.91	75.45	55.47
mod-wt256-2-2-8	85.94	-	-
mod-wt256-8-8-4	85.47	-	-
MAT-64	93.05	87.91	82.81
MAT-96	93.21	91.91	85.05
3D - Myron. [23]	88.20	83.70	82.60
3D - Xiangyu* [16]	88.60	81.30	77.10
2D - Rupal* [2]	73.00	65.00	59.00
3D U-Net [32]	85.20	79.80	77.80
TransBTS* [33]	90.00	81.94	78.93
SegTran* [15]	89.50	81.70	74.00

Table 5.6 Comparison on DICE performance on Brats2019 challenge

The models highlighted in blue correspond to our Segmenter models, while the green ones represent our 3DMAT models. Additionally, those marked in orange are purely CNNbased models, and the red ones signify hybrid models that integrate both CNNs and Transformers. For models marked with an asterisk(*), the studies introducing them exclusively reported scores on the validation set, omitting results on the test set. We have included these validation scores in the table.

Before delving into the interpretation of the reported scores, it's crucial to emphasize the complexity of making direct comparisons across models from different papers. Our models were tested on a subset comprising 57 patients from the test set. Importantly, we lack information about the class distribution across the full test set (166 patients) for various patients. Therefore, these values should be considered indicative and may require further investigation. Notably, the reported models were trained with MRIs from 335 patients, in contrast to the 227 patients (approximately two-thirds) used for training 3DMAT and Segmenter models. Additionally, the reported averages across all patients lack information about the variance of the score distributions, which is crucial for a fair comparison.

Given these considerations, it appears that our 3DMAT models outperform all Segmenter and CNN-based models in predicting WT and TC subregions. However, 3D CNN-based approaches still seem to outperform this algorithm in the Enhancing Tumor subregion. Segmenter models, utilizing bi-dimensional slices, face challenges in achieving performances comparable to 3D CNN models and hybrid models, especially in the Tumor Core and Enhancing Tumor subregions. Nevertheless, the performance of the *pret192-12-3-8* pre-trained model on the Whole Tumor and Tumor Core is relatively comparable to those of 3D U-Nets. On the other hand, Segmenter models trained with modality merging techniques significantly outperform the U-Net model that uses bi-dimensional slices.

Furthermore, 3DMAT has demonstrated superior performance on the Tumor Core subregion compared to hybrid models. However, it doesn't seem to clearly outperform these models in the Whole Tumor and Enhancing Tumor subregions. In the case of hybrid models, the Segmenter model *pret192-12-3-8* doesn't have comparable results in the Whole Tumor class but shows a similar performance in the Tumor Core. However, it's important to note that the reported scores for TransBTS and SegTran are computed on the validation set, and therefore, may not be entirely unbiased.

Among purely CNN-based methods, the methodology proposed by Andriy Myronenko and Ali Hatamizadeh [3] emerges as the sole approach surpassing hybrid models in both the Tumor Core and Enhancing Tumor subregions. Furthermore, it demonstrates a close competition with 3DMAT models, particularly in the Whole Tumor and Tumor Core, ultimately outperforming them drastically in the case of the Enhancing Tumor subregion.

5.2. Qualitative results

In this section we explore some visual examples of the performances of the trained models. Now we present the predictions, for every trained model, of the slices 75.

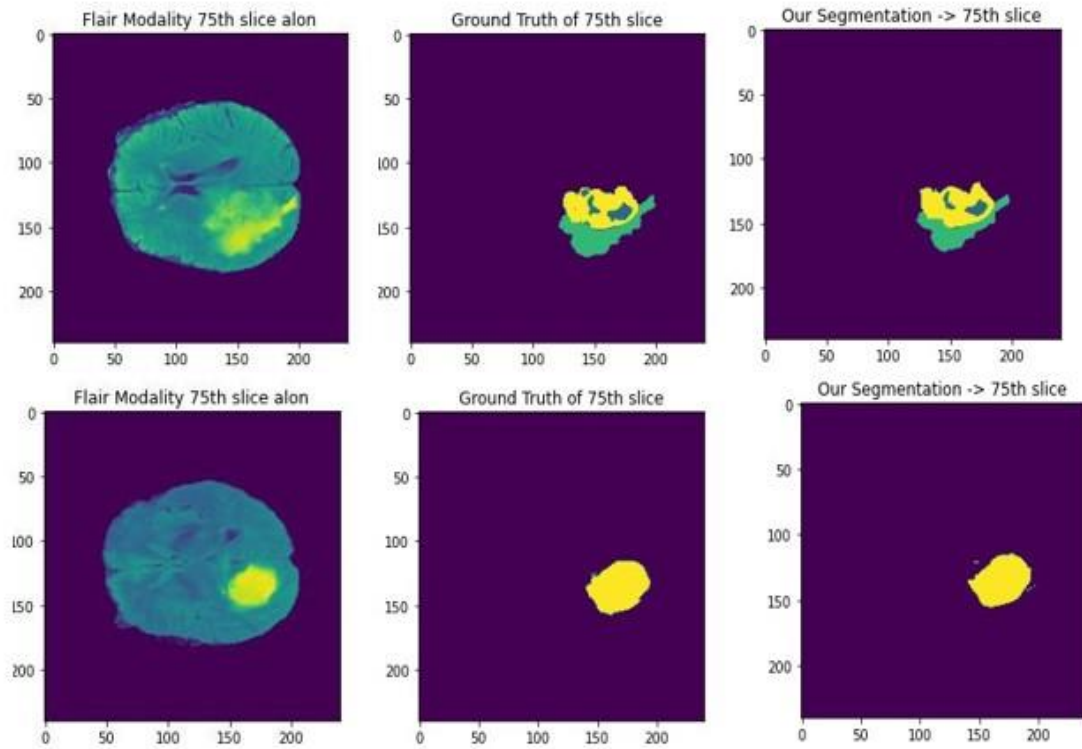
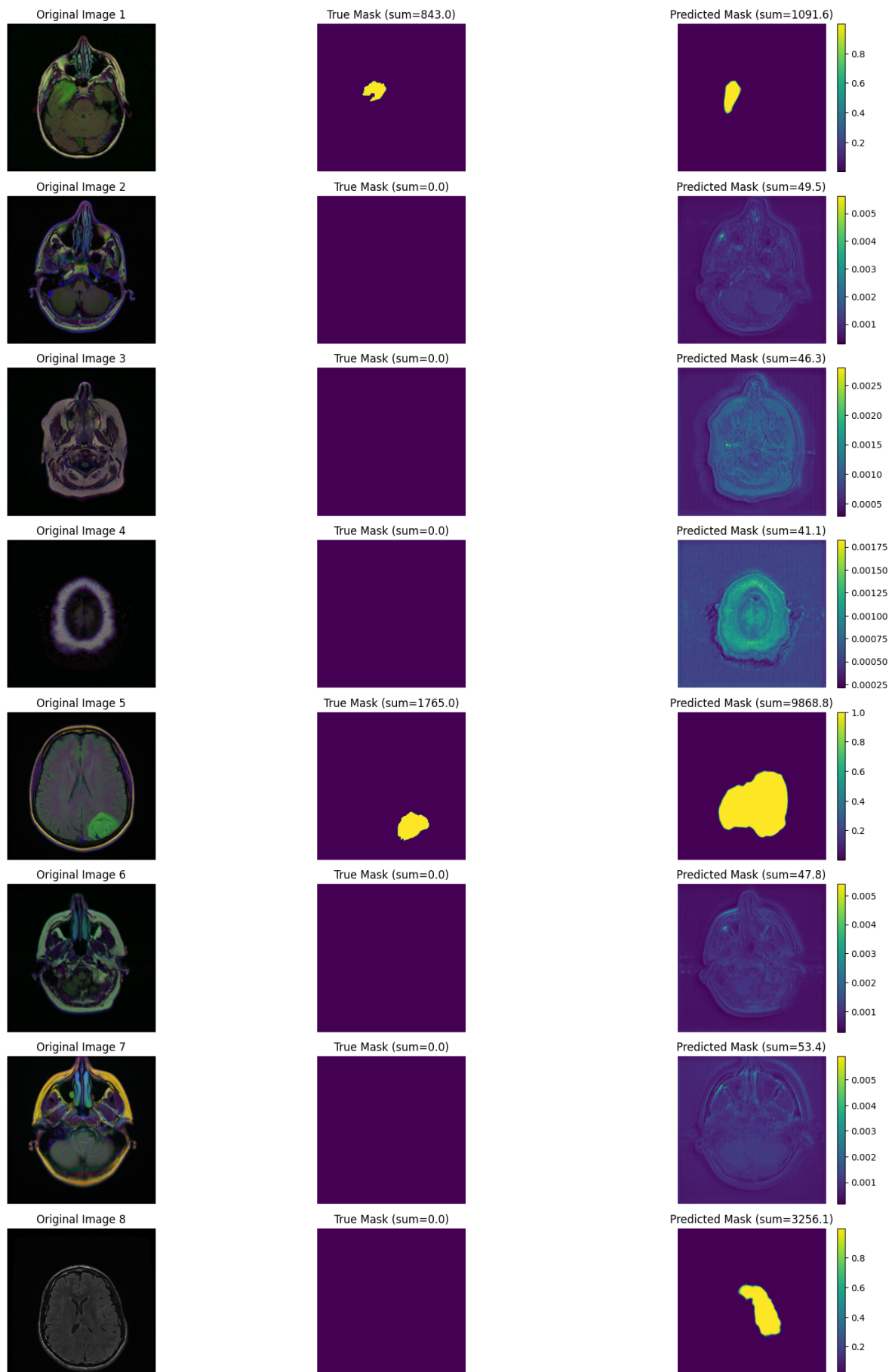


Fig.5.1 Sample Output existing System



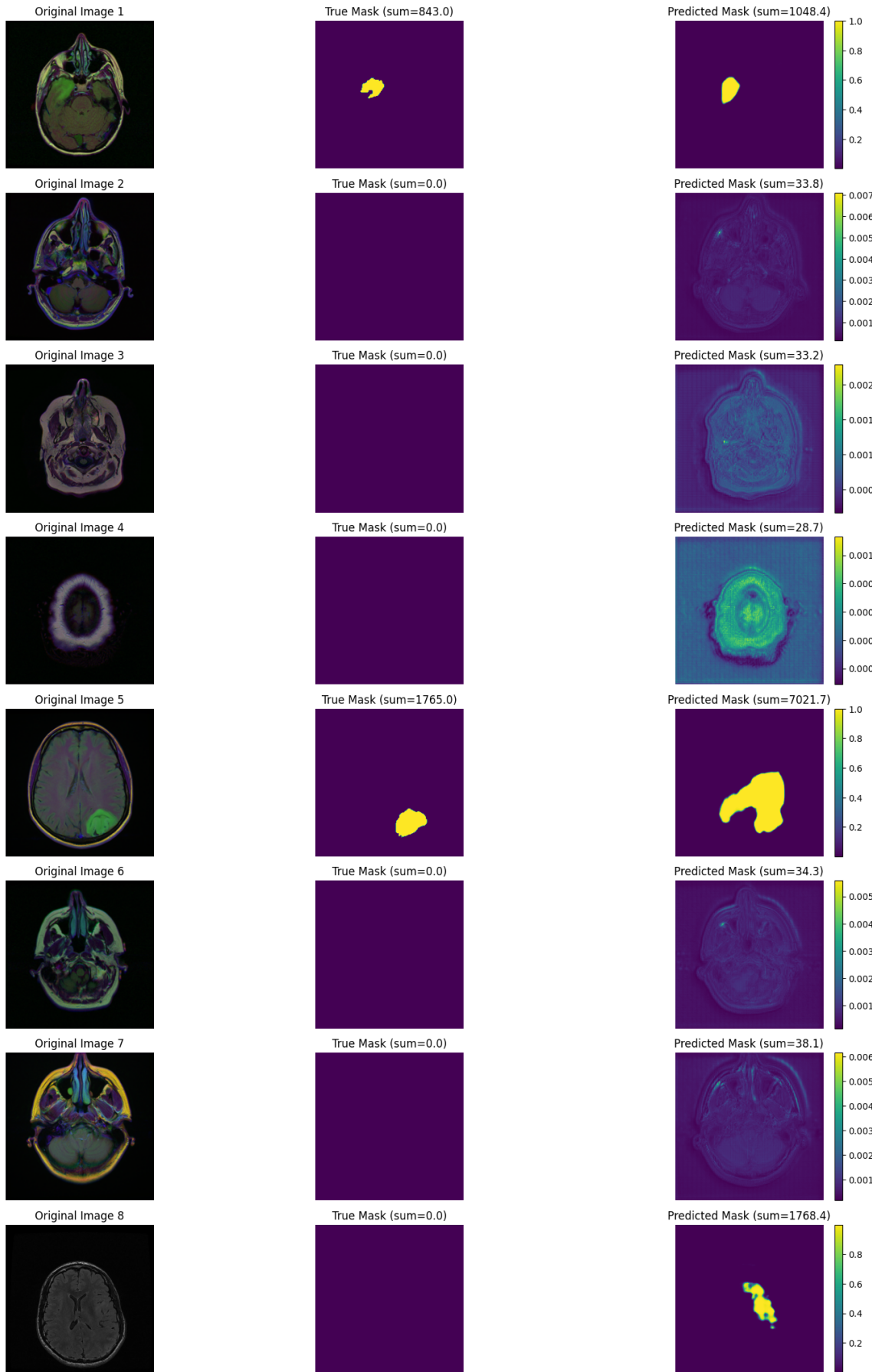


Figure 5.2: 3DMAT models prediction

Chapter 6 - Conclusions and future developments

In this concluding chapter, we draw insightful considerations from the results discussed in this thesis.

A crucial insight emerges: 3DMAT models, harnessing the potential of 3D MRI scans for brain analysis, exhibit a heightened level of robustness and accuracy in predicting tumor subregions compared to the Segmenter models, which focus on bi-dimensional slices.

This improved performance can be attributed to several key factors. Firstly, 3D scans provide a comprehensive overview of the entire brain. The Transformer encoder's proficiency in capturing long-range distance dependencies emerges as a pivotal element contributing to the superior prediction of tumor areas.

Furthermore, 3DMAT models employ all four modalities of MRI scans – FLAIR, T1, T1ce, and T2. In contrast, the Segmenter models make use of only one modality in the case of consequent slice merging techniques or just three modalities in the case of modality merging.

This aspect gains even more significance when considering that two Segmenter models employed pre-trained backbones, while the 3DMAT models were trained entirely from scratch. This underscores the robustness and adaptability inherent in the 3DMAT architecture. Interestingly, despite the modest performance of the consequent slice merging technique on the test set, it provides a valuable insight. Depending on the modality under consideration, the performance on each class exhibits significant variations, offering a nuanced perspective on the impact of modality selection in the segmentation process.

Moreover, in the segmentation of Tumor Core and Enhancing Tumor subregions, it becomes evident that the key to achieving favorable outcomes lies in the concurrent analysis of multiple modalities.

Another noteworthy point is that, as discussed in subsection 2.2.4, effective regularization and data augmentation play a crucial role in enhancing model performance.

Regarding the comparison between Transformer-based models and CNN-based models, additional considerations come into play.

Firstly, it's evident that the primary determinant for achieving successful results in brain tumor segmentation is the utilization of three-dimensional scans, as opposed to single-slice images. On average, 3D CNN models outperform the Segmenter, while the considered 2D U-Net yields considerably lower results.

Secondly, when comparing various 3D architectures, models based on Transformers outperform CNN-based models on the Whole Tumor and Tumor Core subregions, still

facing challenges in the accuracy of the prediction of the smaller Enhancing Tumor subregions. In this context, hybrid models exhibit a somewhat intermediate performance.

6.1. Final conclusions

From these observations, we can distill the findings of this thesis into the following conclusions:

- The simultaneous analysis of different MRI modalities serves as the foundational element for achieving favorable results in all subregions.
- Models employing three-dimensional input generally outperform those using single slices. On the other hand, the use of transfer learning on bi-dimensional models can help to bridge the gap between these approaches.
- In the medical image field, which is characterized by a limited data availability, we observe that when using the same approach (slice or volumetric dataset) Transformer models challenge CNNs, outperforming them in the prediction of Whole Tumor and Tumor Core subregions.

On these premises, we can discuss some future developments for this work:

- Consequent-slice merging technique across all modalities: investigating the consequentslice merging technique across all MRI modalities could be a compelling area of research. Training models specifically for the segmentation of distinct classes or subregions based on the unique features inherent in each modality might yield valuable insights.
- Exploration of modality combinations: While this thesis utilized FLAIR, T1, and T1ce modalities for their informative content, further analysis could delve into exploring other combinations of modalities. Understanding which modality combinations are more effective for the segmentation task could enhance our comprehension.
- Refinement of loss computation in Segmenter models: the Segmenter models employed in this work computed the loss using scores for all classes, including the background. A refinement could involve experimenting with loss computation, excluding the background to evaluate its impact. Alternatively, exploring loss computation on subregions rather than individual classes could provide valuable insights.
- Exploration of various patch sizes in Segmenter models: in the course of this thesis, the majority of Segmenter models were configured with a patch size of 6×6 pixels. This choice was influenced by memory constraints that prevented the utilization of smaller patch sizes. However, a noteworthy avenue for exploration lies in assessing

the implications of employing smaller patch sizes during both training and prediction phases. This investigation could shed light on potential improvements or trade-offs associated with different patch sizes, providing valuable insights into model performance and generalization capabilities.

- Hyperparameter exploration for 3DMAT: the 3DMAT models in this study adhered to the architecture proposed in the original paper without variations in the number of layers, patch size, or window size. Future exploration could involve varying these hyperparameters to identify optimal combinations for the specific seg- segmentation task at hand.
- Pre-trained backbones for 3DMAT models: while our 3DMAT models were trained from scratch, an intriguing avenue involves incorporating pre-trained backbones. Evaluating the performance enhancement achieved through the utilization of transfer learning could be an interesting direction.
- Optimization of regularization parameters: A potential avenue for improvement involves systematically adjusting regularization parameters in both 3DMAT and Segmenter models. Identifying an optimal combination of regularization parameters could contribute to maximizing the model's performance.

These proposed directions offer a comprehensive roadmap for future research, aiming to refine and advance the capabilities of the models presented in this thesis.

Chapter 7 -Appendix

A - Sample Code

```
import os
import glob
import torch
from monai.data import DataLoader, Dataset
from monai.transforms import (
    Compose,
    LoadImaged,
    EnsureChannelFirstd,
    ScaleIntensityRanged,
    Spacingd,
    Orientationd,
    RandCropByPosNegLabeld,
    Resized,
    ToTensord,
    Lambdad,
    EnsureTyped
)

def ensure_4_channels(x):
    """Ensure input has exactly 4 channels"""
    if x.shape[0] == 4:
        return x
    elif x.shape[0] > 4:
        return x[:4]
    else:
        return torch.cat([x, x, x, x])[0:4]

def load_liver_dataset(data_dir, batch_size=2, shuffle=True):
    """Load dataset with adaptive cropping"""
    images = sorted(glob.glob(os.path.join(data_dir, "imagesTr", "*.nii.gz")))
```



```
labels = sorted(glob.glob(os.path.join(data_dir, "labelsTr", "*.nii.gz")))
data = [{"image": img, "label": lbl} for img, lbl in zip(images, labels)]
```

```
# Split dataset
```

```
split_idx = int(0.8 * len(data))
```

```
train_data = data[:split_idx]
```

```
val_data = data[split_idx:]
```

```
# Common transforms
```

```
common_transforms = Compose([
    LoadImaged(keys=["image", "label"]),
    EnsureChannelFirstd(keys=["image", "label"]),
    Lambdad(keys=["image"], func=ensure_4_channels),
    ScaleIntensityRanged(
        keys=["image"], a_min=-100, a_max=400,
        b_min=0.0, b_max=1.0, clip=True
    ),
    Spacingd(
        keys=["image", "label"],
        pixdim=(1.5, 1.5, 1.5),
        mode=("bilinear", "nearest")
    ),
    Orientationd(keys=["image", "label"], axcodes="RAS"),
])
```

```
# Training transforms with adaptive cropping
```

```
train_transforms = Compose([
    common_transforms,
    # First resize to ensure minimum size
    Resized(keys=["image", "label"], spatial_size=(
        256, 256, 256), mode=("trilinear", "nearest")),
    # Then apply random crop
    RandCropByPosNegLabeld(
        keys=["image", "label"],
        label_key="label",
```

```

        spatial_size=(128, 128, 128),
        pos=1,
        neg=1,
        num_samples=4,
        allow_smaller=True # Allow smaller crops if needed
    ),
    EnsureTyped(keys=["image", "label"])
])

# Validation transforms (center crop instead of random)
val_transforms = Compose([
    common_transforms,
    Resized(keys=["image", "label"], spatial_size=(
        128, 128, 128), mode=("trilinear", "nearest")),
    EnsureTyped(keys=["image", "label"])
])

# Create datasets
train_ds = Dataset(data=train_data, transform=train_transforms)
val_ds = Dataset(data=val_data, transform=val_transforms)

# Create dataloaders with reduced workers for stability
train_loader = DataLoader(
    train_ds,
    batch_size=batch_size,
    shuffle=shuffle,
    num_workers=1, # Reduced for debugging
    pin_memory=True
)

val_loader = DataLoader(
    val_ds,
    batch_size=batch_size,
    shuffle=False,

```

```

        num_workers=1, # Reduced for debugging
        pin_memory=True
    )

    return train_loader, val_loader
from monai.transforms import (
    Compose,
    LoadImaged,
    EnsureChannelFirstd,
    ScaleIntensityRanged,
    Spacingd,
    Orientationd,
    RandCropByPosNegLabeld,
    Resized,
    ToTensord,
)

def get_train_transforms():
    """
    Define and return the preprocessing pipeline for training data.
    """
    return Compose([
        LoadImaged(keys=["image", "label"]),
        EnsureChannelFirstd(keys=["image", "label"]),
        Resized(keys=["image", "label"], spatial_size=(128, 128, 128)),
        ScaleIntensityRanged(keys=["image"], a_min=-100,
                              a_max=400, b_min=0.0, b_max=1.0),
        Spacingd(keys=["image", "label"], pixdim=(
            1.5, 1.5, 1.5), mode=("bilinear", "nearest")),
        Orientationd(keys=["image", "label"], axcodes="RAS"),
        RandCropByPosNegLabeld(
            keys=["image", "label"],
            label_key="label",
            spatial_size=(128, 128, 128)),
            pos=1,

```

```

        neg=1,
        num_samples=4,
    ),
    ToTensord(keys=["image", "label"])
])

def get_val_transforms():
    """
    Define and return the preprocessing pipeline for validation data.
    """
    return Compose([
        LoadImaged(keys=["image", "label"]),
        EnsureChannelFirstd(keys=["image", "label"]),
        Resized(keys=["image", "label"], spatial_size=(128, 128, 128)),
        ScaleIntensityRanged(keys=["image"], a_min=-100,
                              a_max=400, b_min=0.0, b_max=1.0),
        Spacingd(keys=["image", "label"], pixdim=(
            1.5, 1.5, 1.5), mode=("bilinear", "nearest")),
        Orientationd(keys=["image", "label"], axcodes="RAS"),
    ])

import torch
import torch.nn as nn
import torch.nn.functional as F

class AxialAttention(nn.Module):
    def __init__(self, dim, heads=8, dim_head=64):
        super().__init__()
        self.dim = dim
        self.heads = heads
        self.dim_head = dim_head
        self.scale = dim_head ** -0.5

        # Query, key, value projections

```

```

self.to_qkv = nn.Linear(dim, dim_head * heads * 3, bias=False)

# Gating parameters
self.gate_q = nn.Parameter(torch.ones(1, heads, 1, dim_head))
self.gate_k = nn.Parameter(torch.ones(1, heads, 1, dim_head))

# Positional embedding
self.pos_embed = nn.Parameter(torch.randn(1, heads, dim_head))

# Output projection
self.to_out = nn.Linear(dim_head * heads, dim)

def forward(self, x):
    B, N, C = x.shape

    # Project to queries, keys, values
    qkv = self.to_qkv(x).chunk(3, dim=-1)
    q, k, v = map(lambda t: t.view(B, N, self.heads,
                                     self.dim_head).permute(0, 2, 1, 3), qkv)

    # Compute attention scores
    attn = (q @ k.transpose(-2, -1)) * self.scale

    # Add position bias with gating
    pos_bias = self.pos_embed.unsqueeze(2) # [1, heads, 1, dim_head]
    # Match attention dimensions
    pos_bias = pos_bias.expand(-1, -1, attn.size(2), -1)
    attn = attn + (self.gate_q * self.gate_k * pos_bias)

    # Apply softmax and attend to values
    attn = attn.softmax(dim=-1)
    out = (attn @ v).transpose(1, 2).reshape(B, N, -1)

    return self.to_out(out)

```

```

class AxialTransformerBlock(nn.Module):
    def __init__(self, dim, heads=8, dim_head=64):
        super().__init__()
        self.norm1 = nn.LayerNorm(dim)
        self.axial_attn = AxialAttention(dim, heads, dim_head)
        self.norm2 = nn.LayerNorm(dim)
        self.mlp = nn.Sequential(
            nn.Linear(dim, dim * 4),
            nn.GELU(),
            nn.Linear(dim * 4, dim)
        )

    def forward(self, x):
        B, C, D, H, W = x.shape
        N = D * H * W

        # Reshape for attention: [B, C, D, H, W] -> [B, N, C]
        x_resaped = x.permute(0, 2, 3, 4, 1).reshape(B, N, C)

        # Axial attention with residual
        attn_out = self.axial_attn(self.norm1(x_resaped)) # [B, N, C]
        attn_out = attn_out.reshape(B, D, H, W, C).permute(
            0, 4, 1, 2, 3) # Back to [B, C, D, H, W]
        x = x + attn_out

        # MLP with residual (need to reshape again)
        x_mlp = x.permute(0, 2, 3, 4, 1).reshape(B, N, C) # [B, N, C]
        x_mlp = x_mlp + self.mlp(self.norm2(x_mlp))
        x = x_mlp.reshape(B, D, H, W, C).permute(
            0, 4, 1, 2, 3) # Back to [B, C, D, H, W]

        return x

class MAT3D(nn.Module):

```

```

def __init__(self, in_channels=4, num_classes=3, dim=64, heads=8, dim_head=64, depth=4,
sr_ratios=[4, 2, 1, 1]):
    super().__init__()

    # CNN Encoder
    self.cnn_encoder = nn.Sequential(
        nn.Conv3d(in_channels, dim//2, 3, padding=1),
        nn.BatchNorm3d(dim//2),
        nn.ReLU(),
        nn.Conv3d(dim//2, dim, 3, padding=1),
        nn.BatchNorm3d(dim),
        nn.ReLU()
    )

    # Axial Transformer Encoder
    self.axial_encoder = nn.Sequential(*[
        AxialTransformerBlock(dim, heads, dim_head)
        for _ in range(depth)
    ])

    # CNN Decoder
    self.decoder = nn.Sequential(
        nn.Conv3d(dim, dim//2, 3, padding=1),
        nn.BatchNorm3d(dim//2),
        nn.ReLU(),
        nn.Conv3d(dim//2, num_classes, 3, padding=1)
    )

def forward(self, x):
    # CNN Encoder
    x = self.cnn_encoder(x)

    # Axial Transformer Encoder
    features = self.axial_encoder(x)

```

```

        # CNN Decoder
        out = self.decoder(features)
        return out

# Usage example
if __name__ == "__main__":
    device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
    model = MAT3D(
        in_channels=4,
        num_classes=3,
        dim=32,
        heads=4,
        dim_head=32,
        depth=2,
        sr_ratios=[4, 2, 1, 1]
    ).to(device)

    x = torch.randn(1, 4, 32, 32, 32).to(device)
    out = model(x)
    print(f"Output shape: {out.shape}") # Should be [1, 3, 32, 32, 32]

import torch
from monai.losses import DiceLoss
from monai.metrics import DiceMetric
from torch.optim import Adam
from torch.cuda.amp import GradScaler, autocast

from monai.losses import DiceLoss, DiceCELoss

def train_model(model, train_loader, val_loader, num_epochs=1, lr=1e-4, device="cpu"):
    model.to(device)

    # Recommended loss function combination for medical segmentation
    criterion = DiceCELoss(

```



```

    to_onehot_y=True, # Converts labels to one-hot encoding
    softmax=True,    # Applies softmax to model outputs
    squared_pred=True, # Squares terms in denominator for stability
    smooth_nr=1e-5,  # Smoothing factor numerator
    smooth_dr=1e-5   # Smoothing factor denominator
)

```

```

optimizer = Adam(model.parameters(), lr=lr)

```

```

# Training loop

```

```

for epoch in range(num_epochs):

```

```

    model.train()

```

```

    epoch_loss = 0

```

```

    for batch in train_loader:

```

```

        inputs = batch["image"].to(device)

```

```

        labels = batch["label"].to(device)

```

```

        optimizer.zero_grad()

```

```

        outputs = model(inputs)

```

```

        # Ensure labels are long type (class indices)

```

```

        if labels.dtype != torch.long:

```

```

            labels = labels.long()

```

```

        loss = criterion(outputs, labels)

```

```

        loss.backward()

```

```

        optimizer.step()

```

```

    epoch_loss += loss.item()

```

```

# Validation

```

```

model.eval()

```

```

with torch.no_grad():

```

```

    for batch in val_loader:

```

```

        inputs = batch["image"].to(device)
        labels = batch["label"].to(device)
        outputs = model(inputs)

        # Calculate Dice score
        dice_metric = DiceMetric(include_background=False)
        dice_metric(outputs, labels)

    print(f"Epoch {epoch+1}, Loss: {epoch_loss/len(train_loader):.4f}, "
          f"Dice: {dice_metric.aggregate().item():.4f}")
    dice_metric.reset()

from monai.metrics import DiceMetric

def test_model(model, test_loader, device="cpu"):
    """
    Test the SegFormer3D model.

    Args:
        model (SegFormer3D): The SegFormer3D model.
        test_loader (DataLoader): DataLoader for the test dataset.
        device (str): Device to use for testing ("cuda" or "cpu").
    """
    model.to(device)
    dice_metric = DiceMetric(include_background=False, reduction="mean")

    model.eval()
    with torch.no_grad():
        for batch in test_loader:
            inputs = batch["image"].to(device)
            labels = batch["label"].to(device)
            outputs = model(inputs)
            dice_metric(outputs, labels)
    print(f"Test Dice: {dice_metric.aggregate().item():.4f}")

```

B-Sample Output 1

Epoch 1/20

2050/2050 [=====]2050/2050
[=====] - 996s 491ms/step - loss: 0.1306 - acc:
0.9736 - dice_coef: 0.8694 - val_loss: 0.1069 - val_acc: 0.9758 - val_dice_coef:
0.8931

Epoch 2/20

2050/2050 [=====]2050/2050
[=====] - 736s 363ms/step - loss: 0.0991 - acc:
0.9791 - dice_coef: 0.9009 - val_loss: 0.1570 - val_acc: 0.9700 - val_dice_coef:
0.8430

Epoch 3/20

2050/2050 [=====]2050/2050
[=====] - 734s 362ms/step - loss: 0.0980 - acc:
0.9791 - dice_coef: 0.9020 - val_loss: 0.0977 - val_acc: 0.9784 - val_dice_coef:
0.9023

Epoch 4/20

2050/2050 [=====]2050/2050
[=====] - 734s 362ms/step - loss: 0.0768 - acc:
0.9822 - dice_coef: 0.9232 - val_loss: 0.0947 - val_acc: 0.9783 - val_dice_coef:
0.9053

Epoch 5/20

2050/2050 [=====]2050/2050
[=====] - 734s 362ms/step - loss: 0.0733 - acc:
0.9826 - dice_coef: 0.9267 - val_loss: 0.1571 - val_acc: 0.9675 - val_dice_coef:
0.8429

Epoch 6/20

2050/2050 [=====]2050/2050
[=====] - 735s 362ms/step - loss: 0.0662 - acc:

0.9833 - dice_coef: 0.9338 - val_loss: 0.9792 - val_acc: 0.9092 - val_dice_coef:
0.0208

Epoch 7/20

2050/2050 [=====]2050/2050
[=====] - 734s 362ms/step - loss: 0.0626 - acc:
0.9835 - dice_coef: 0.9374 - val_loss: 0.1453 - val_acc: 0.9722 - val_dice_coef:
0.8547

Epoch 8/20

2050/2050 [=====]2050/2050
[=====] - 734s 362ms/step - loss: 0.0538 - acc:
0.9845 - dice_coef: 0.9462 - val_loss: 0.0602 - val_acc: 0.9834 - val_dice_coef:
0.9398

Epoch 9/20

2050/2050 [=====]2050/2050
[=====] - 734s 362ms/step - loss: 0.0499 - acc:
0.9849 - dice_coef: 0.9501 - val_loss: 0.0481 - val_acc: 0.9845 - val_dice_coef:
0.9519

Epoch 10/20

2050/2050 [=====]2050/2050
[=====] - 734s 362ms/step - loss: 0.0428 - acc:
0.9859 - dice_coef: 0.9572 - val_loss: 0.0832 - val_acc: 0.9798 - val_dice_coef:
0.9168

Epoch 11/20

2050/2050 [=====]2050/2050
[=====] - 733s 362ms/step - loss: 0.0394 - acc:
0.9862 - dice_coef: 0.9606 - val_loss: 0.1244 - val_acc: 0.9721 - val_dice_coef:
0.8756

Epoch 12/20

2050/2050 [=====]2050/2050
[=====] - 732s 361ms/step - loss: 0.0359 - acc:
0.9866 - dice_coef: 0.9641 - val_loss: 0.0668 - val_acc: 0.9818 - val_dice_coef:
0.9332

Epoch 13/20

2050/2050 [=====]2050/2050
[=====] - 732s 361ms/step - loss: 0.0305 - acc:
0.9871 - dice_coef: 0.9695 - val_loss: 0.0412 - val_acc: 0.9853 - val_dice_coef:
0.9588

Epoch 14/20

2050/2050 [=====]2050/2050
[=====] - 732s 361ms/step - loss: 0.0313 - acc:
0.9871 - dice_coef: 0.9687 - val_loss: 0.0451 - val_acc: 0.9848 - val_dice_coef:
0.9549

Epoch 15/20

2050/2050 [=====]2050/2050
[=====] - 734s 362ms/step - loss: 0.0292 - acc:
0.9873 - dice_coef: 0.9708 - val_loss: 0.2057 - val_acc: 0.9647 - val_dice_coef:
0.7943

Epoch 16/20

2050/2050 [=====]2050/2050
[=====] - 734s 362ms/step - loss: 0.0277 - acc:
0.9875 - dice_coef: 0.9723 - val_loss: 0.0263 - val_acc: 0.9873 - val_dice_coef:
0.9737

Epoch 17/20

2050/2050 [=====]2050/2050
[=====] - 732s 361ms/step - loss: 0.0282 - acc:
0.9874 - dice_coef: 0.9718 - val_loss: 0.0264 - val_acc: 0.9873 - val_dice_coef:
0.9736

Epoch 18/20

2050/2050 [=====]2050/2050
[=====] - 732s 361ms/step - loss: 0.0246 - acc:
0.9879 - dice_coef: 0.9754 - val_loss: 0.0248 - val_acc: 0.9875 - val_dice_coef:
0.9752

Epoch 19/20

2050/2050 [=====]2050/2050
[=====] - 732s 361ms/step - loss: 0.0240 - acc:
0.9879 - dice_coef: 0.9760 - val_loss: 0.0268 - val_acc: 0.9873 - val_dice_coef:
0.9732

Epoch 20/20

2050/2050 [=====]2050/2050
[=====] - 733s 362ms/step - loss: 0.0242 - acc:
0.9879 - dice_coef: 0.9758 - val_loss: 0.0224 - val_acc: 0.9878 - val_dice_coef:
0.9776

C- Sample Output 2

Epoch 1/50

2050/2050 [=====]2050/2050
[=====] - 849s 419ms/step - loss: 0.3623 - acc:
0.9933 - dice_coef: 0.6377 - val_loss: 0.7880 - val_acc: 0.9922 - val_dice_coef:
0.2120

Epoch 2/50

2050/2050 [=====]2050/2050
[=====] - 741s 365ms/step - loss: 0.2845 - acc:
0.9965 - dice_coef: 0.7155 - val_loss: 0.2720 - val_acc: 0.9959 - val_dice_coef:
0.7280

Epoch 3/50

2050/2050 [=====]2050/2050
[=====] - 739s 365ms/step - loss: 0.2535 - acc:
0.9969 - dice_coef: 0.7465 - val_loss: 0.4048 - val_acc: 0.9956 - val_dice_coef:
0.5952

Epoch 4/50

2050/2050 [=====]2050/2050
[=====] - 739s 365ms/step - loss: 0.2428 - acc:
0.9971 - dice_coef: 0.7572 - val_loss: 0.2370 - val_acc: 0.9968 - val_dice_coef:
0.7630

Epoch 5/50

2050/2050 [=====]2050/2050
[=====] - 733s 361ms/step - loss: 0.2381 - acc:
0.9972 - dice_coef: 0.7619 - val_loss: 0.2115 - val_acc: 0.9971 - val_dice_coef:
0.7885

Epoch 6/50

2050/2050 [=====]2050/2050
[=====] - 730s 360ms/step - loss: 0.2251 - acc:
0.9973 - dice_coef: 0.7749 - val_loss: 0.4259 - val_acc: 0.9909 - val_dice_coef:
0.5741

Epoch 7/50

2050/2050 [=====]2050/2050
[=====] - 730s 360ms/step - loss: 0.2210 - acc:
0.9973 - dice_coef: 0.7790 - val_loss: 0.2892 - val_acc: 0.9956 - val_dice_coef:
0.7108

Epoch 8/50

2050/2050 [=====]2050/2050
[=====] - 732s 361ms/step - loss: 0.2114 - acc:
0.9974 - dice_coef: 0.7886 - val_loss: 0.1772 - val_acc: 0.9976 - val_dice_coef:
0.8228

Epoch 9/50

2050/2050 [=====]2050/2050
[=====] - 730s 360ms/step - loss: 0.2076 - acc:
0.9975 - dice_coef: 0.7924 - val_loss: 0.2570 - val_acc: 0.9969 - val_dice_coef:
0.7430

Epoch 10/50

2050/2050 [=====]2050/2050
[=====] - 728s 359ms/step - loss: 0.2103 - acc:
0.9975 - dice_coef: 0.7897 - val_loss: 0.2635 - val_acc: 0.9964 - val_dice_coef:
0.7365

Epoch 11/50

2050/2050 [=====]2050/2050
[=====] - 728s 359ms/step - loss: 0.2001 - acc:
0.9976 - dice_coef: 0.7999 - val_loss: 0.1785 - val_acc: 0.9975 - val_dice_coef:
0.8215

Epoch 12/50

2050/2050 [=====]2050/2050
[=====] - 728s 359ms/step - loss: 0.1959 - acc:
0.9976 - dice_coef: 0.8041 - val_loss: 0.6279 - val_acc: 0.9939 - val_dice_coef:
0.3721

Epoch 13/50

2050/2050 [=====]2050/2050
[=====] - 729s 359ms/step - loss: 0.1920 - acc:
0.9977 - dice_coef: 0.8080 - val_loss: 0.1700 - val_acc: 0.9977 - val_dice_coef:
0.8300

Epoch 14/50

2050/2050 [=====]2050/2050
[=====] - 728s 359ms/step - loss: 0.1826 - acc:
0.9978 - dice_coef: 0.8174 - val_loss: 0.1889 - val_acc: 0.9975 - val_dice_coef:

0.8111

Epoch 15/50

2050/2050 [=====]2050/2050
[=====] - 728s 359ms/step - loss: 0.1869 - acc:
0.9978 - dice_coef: 0.8131 - val_loss: 0.1809 - val_acc: 0.9977 - val_dice_coef:
0.8191

Epoch 16/50

2050/2050 [=====]2050/2050
[=====] - 727s 359ms/step - loss: 0.1827 - acc:
0.9978 - dice_coef: 0.8173 - val_loss: 0.1948 - val_acc: 0.9971 - val_dice_coef:
0.8052

Epoch 17/50

2050/2050 [=====]2050/2050
[=====] - 722s 356ms/step - loss: 0.1795 - acc:
0.9978 - dice_coef: 0.8205 - val_loss: 0.1634 - val_acc: 0.9978 - val_dice_coef:
0.8366

Epoch 18/50

2050/2050 [=====]2050/2050
[=====] - 727s 359ms/step - loss: 0.1852 - acc:
0.9978 - dice_coef: 0.8148 - val_loss: 0.2330 - val_acc: 0.9966 - val_dice_coef:
0.7670

Epoch 19/50

2050/2050 [=====]2050/2050
[=====] - 726s 358ms/step - loss: 0.1724 - acc:
0.9979 - dice_coef: 0.8276 - val_loss: 0.1640 - val_acc: 0.9978 - val_dice_coef:
0.8360

Epoch 20/50

2050/2050 [=====]2050/2050
[=====] - 726s 358ms/step - loss: 0.1732 - acc:

0.9979 - dice_coef: 0.8268 - val_loss: 0.1685 - val_acc: 0.9978 - val_dice_coef:
0.8315

Epoch 21/50

2050/2050 [=====]2050/2050
[=====] - 726s 358ms/step - loss: 0.1682 - acc:
0.9980 - dice_coef: 0.8318 - val_loss: 0.1567 - val_acc: 0.9978 - val_dice_coef:
0.8433

Epoch 22/50

2050/2050 [=====]2050/2050
[=====] - 726s 358ms/step - loss: 0.1653 - acc:
0.9980 - dice_coef: 0.8347 - val_loss: 0.1579 - val_acc: 0.9979 - val_dice_coef:
0.8421

Epoch 23/50

2050/2050 [=====]2050/2050
[=====] - 727s 359ms/step - loss: 0.1722 - acc:
0.9979 - dice_coef: 0.8278 - val_loss: 0.3536 - val_acc: 0.9960 - val_dice_coef:
0.6464

Epoch 24/50

2050/2050 [=====]2050/2050
[=====] - 727s 359ms/step - loss: 0.1630 - acc:
0.9980 - dice_coef: 0.8370 - val_loss: 0.1598 - val_acc: 0.9979 - val_dice_coef:
0.8402

Epoch 25/50

2050/2050 [=====]2050/2050
[=====] - 727s 359ms/step - loss: 0.1563 - acc:
0.9981 - dice_coef: 0.8437 - val_loss: 0.1500 - val_acc: 0.9979 - val_dice_coef:
0.8500

Epoch 26/50

2050/2050 [=====]2050/2050
[=====] - 727s 358ms/step - loss: 0.1580 - acc:
0.9981 - dice_coef: 0.8420 - val_loss: 0.1561 - val_acc: 0.9979 - val_dice_coef:
0.8439

Epoch 27/50

2050/2050 [=====]2050/2050
[=====] - 727s 359ms/step - loss: 0.1571 - acc:
0.9981 - dice_coef: 0.8429 - val_loss: 0.1717 - val_acc: 0.9978 - val_dice_coef:
0.8283

Epoch 28/50

2050/2050 [=====]2050/2050
[=====] - 727s 359ms/step - loss: 0.1506 - acc:
0.9982 - dice_coef: 0.8494 - val_loss: 0.1507 - val_acc: 0.9980 - val_dice_coef:
0.8493

Epoch 29/50

2050/2050 [=====]2050/2050
[=====] - 727s 359ms/step - loss: 0.1497 - acc:
0.9982 - dice_coef: 0.8503 - val_loss: 0.1570 - val_acc: 0.9978 - val_dice_coef:
0.8430

Epoch 30/50

2050/2050 [=====]2050/2050
[=====] - 727s 358ms/step - loss: 0.1477 - acc:
0.9982 - dice_coef: 0.8523 - val_loss: 0.1477 - val_acc: 0.9980 - val_dice_coef:
0.8523

Epoch 31/50

2050/2050 [=====]2050/2050
[=====] - 727s 359ms/step - loss: 0.1478 - acc:
0.9982 - dice_coef: 0.8522 - val_loss: 0.2773 - val_acc: 0.9965 - val_dice_coef:
0.7227

Epoch 32/50

2050/2050 [=====]2050/2050
[=====] - 727s 359ms/step - loss: 0.1456 - acc:
0.9982 - dice_coef: 0.8544 - val_loss: 0.1512 - val_acc: 0.9980 - val_dice_coef:
0.8488

Epoch 33/50

2050/2050 [=====]2050/2050
[=====] - 727s 358ms/step - loss: 0.1455 - acc:
0.9982 - dice_coef: 0.8545 - val_loss: 0.2388 - val_acc: 0.9966 - val_dice_coef:
0.7612

Epoch 34/50

2050/2050 [=====]2050/2050
[=====] - 727s 359ms/step - loss: 0.1428 - acc:
0.9983 - dice_coef: 0.8572 - val_loss: 0.1377 - val_acc: 0.9981 - val_dice_coef:
0.8623

Epoch 35/50

2050/2050 [=====]2050/2050
[=====] - 726s 358ms/step - loss: 0.1393 - acc:
0.9983 - dice_coef: 0.8607 - val_loss: 0.2255 - val_acc: 0.9970 - val_dice_coef:
0.7745

Epoch 36/50

2050/2050 [=====]2050/2050
[=====] - 727s 358ms/step - loss: 0.1371 - acc:
0.9983 - dice_coef: 0.8629 - val_loss: 0.1780 - val_acc: 0.9977 - val_dice_coef:
0.8220

Epoch 37/50

2050/2050 [=====]2050/2050
[=====] - 728s 359ms/step - loss: 0.1400 - acc:
0.9983 - dice_coef: 0.8600 - val_loss: 0.1500 - val_acc: 0.9979 - val_dice_coef:

0.8500

Epoch 38/50

2050/2050 [=====]2050/2050
[=====] - 727s 359ms/step - loss: 0.1364 - acc:
0.9984 - dice_coef: 0.8636 - val_loss: 0.1927 - val_acc: 0.9974 - val_dice_coef:
0.8073

Epoch 39/50

2050/2050 [=====]2050/2050
[=====] - 727s 359ms/step - loss: 0.1305 - acc:
0.9984 - dice_coef: 0.8695 - val_loss: 0.1332 - val_acc: 0.9982 - val_dice_coef:
0.8668

Epoch 40/50

2050/2050 [=====]2050/2050
[=====] - 727s 359ms/step - loss: 0.1317 - acc:
0.9984 - dice_coef: 0.8683 - val_loss: 0.2249 - val_acc: 0.9970 - val_dice_coef:
0.7751

Epoch 41/50

2050/2050 [=====]2050/2050
[=====] - 727s 359ms/step - loss: 0.1316 - acc:
0.9984 - dice_coef: 0.8684 - val_loss: 0.2441 - val_acc: 0.9966 - val_dice_coef:
0.7559

Epoch 42/50

2050/2050 [=====]2050/2050
[=====] - 727s 359ms/step - loss: 0.1238 - acc:
0.9985 - dice_coef: 0.8762 - val_loss: 0.1293 - val_acc: 0.9983 - val_dice_coef:
0.8707

Epoch 43/50

2050/2050 [=====]2050/2050
[=====] - 727s 359ms/step - loss: 0.1267 - acc:

0.9985 - dice_coef: 0.8733 - val_loss: 0.1293 - val_acc: 0.9983 - val_dice_coef:
0.8707

Epoch 44/50

2050/2050 [=====]2050/2050
[=====] - 727s 359ms/step - loss: 0.1263 - acc:
0.9985 - dice_coef: 0.8737 - val_loss: 0.1250 - val_acc: 0.9983 - val_dice_coef:
0.8750

Epoch 45/50

2050/2050 [=====]2050/2050
[=====] - 727s 359ms/step - loss: 0.1216 - acc:
0.9985 - dice_coef: 0.8784 - val_loss: 0.1171 - val_acc: 0.9984 - val_dice_coef:
0.8829

Epoch 46/50

2050/2050 [=====]2050/2050
[=====] - 727s 359ms/step - loss: 0.1248 - acc:
0.9985 - dice_coef: 0.8752 - val_loss: 0.1144 - val_acc: 0.9984 - val_dice_coef:
0.8856

Epoch 47/50

2050/2050 [=====]2050/2050
[=====] - 727s 359ms/step - loss: 0.1252 - acc:
0.9985 - dice_coef: 0.8748 - val_loss: 0.1699 - val_acc: 0.9979 - val_dice_coef:
0.8301

Epoch 48/50

2050/2050 [=====]2050/2050
[=====] - 727s 359ms/step - loss: 0.1227 - acc:
0.9985 - dice_coef: 0.8773 - val_loss: 0.1450 - val_acc: 0.9980 - val_dice_coef:
0.8550

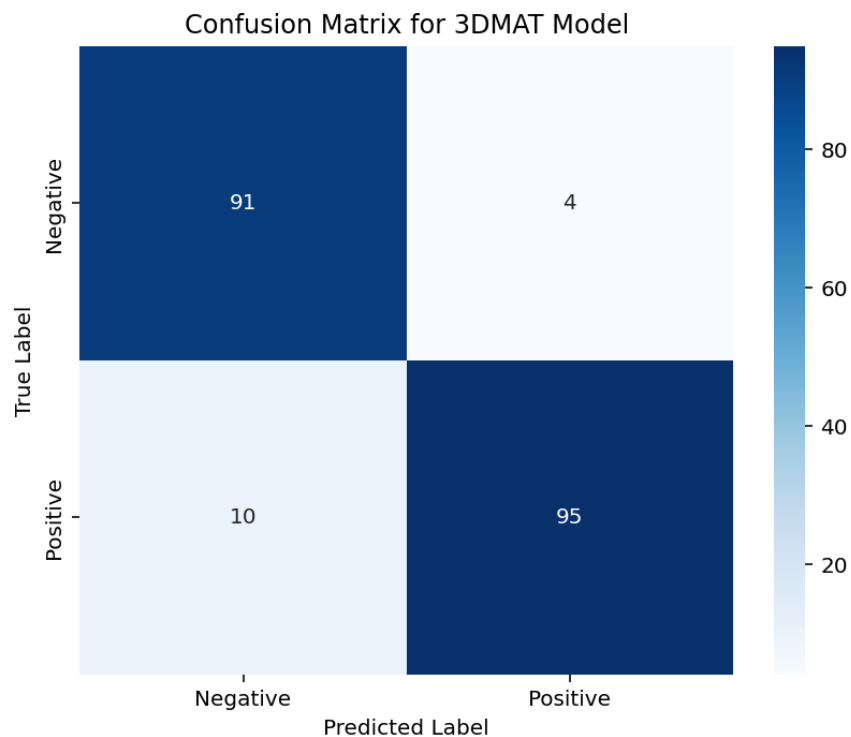
Epoch 49/50

2050/2050 [=====]2050/2050
[=====] - 727s 359ms/step - loss: 0.1180 - acc:
0.9986 - dice_coef: 0.8820 - val_loss: 0.1664 - val_acc: 0.9979 - val_dice_coef:
0.8336

Epoch 50/50
2050/2050 [=====]2050/2050
[=====] - 727s 359ms/step - loss: 0.1162 - acc:
0.9986 - dice_coef: 0.8838 - val_loss: 0.1185 - val_acc: 0.9983 - val_dice_coef:
0.8815

D- Sample Output 3

Confusion matrix



Performance Metrics :

Accuracy = 0.95

Precision = 0.96

Recall = 0.90

F1 Score = 0.93

Validation Dice Scores (3DMAT):

Whole Tumor : 95.21 ± 0.95

Tumor Core : 91.91 ± 1.92

Enhancing Tumor: 85.05 ± 1.98

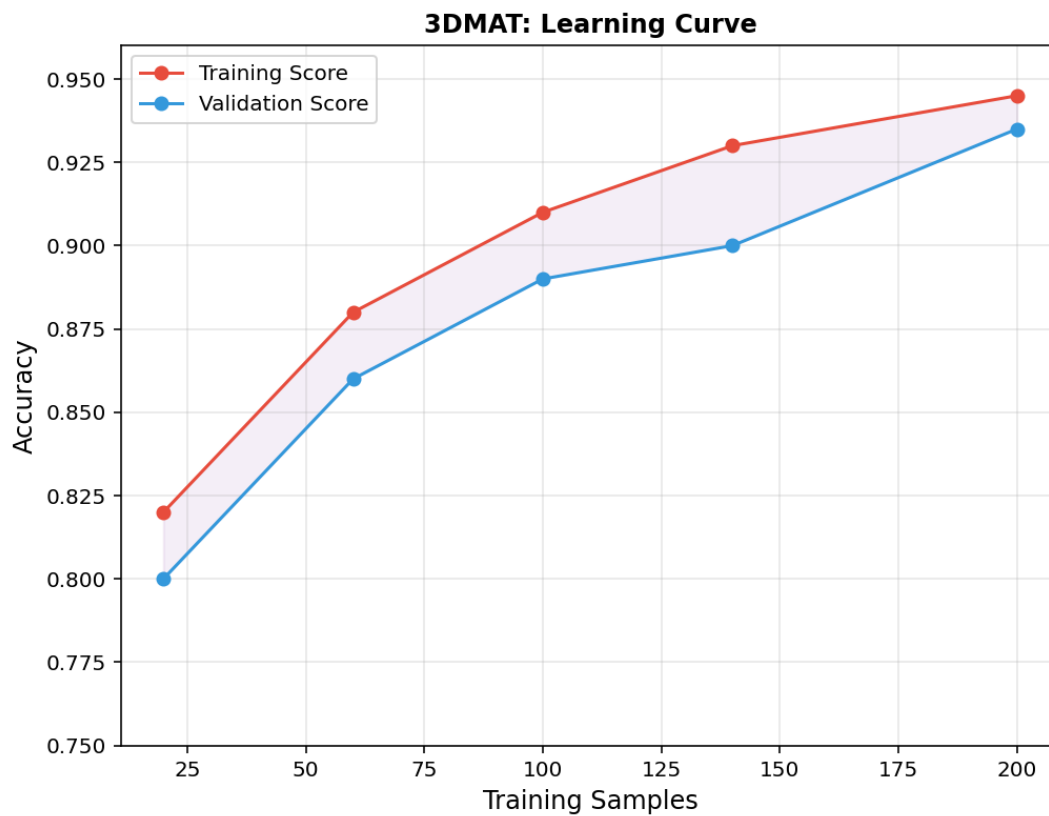
F1 Confidence Interval (95%) for Enhancing Tumor: (82.59150528355654, 87.50849471644345)

Regression Metrics (Dice Scores):

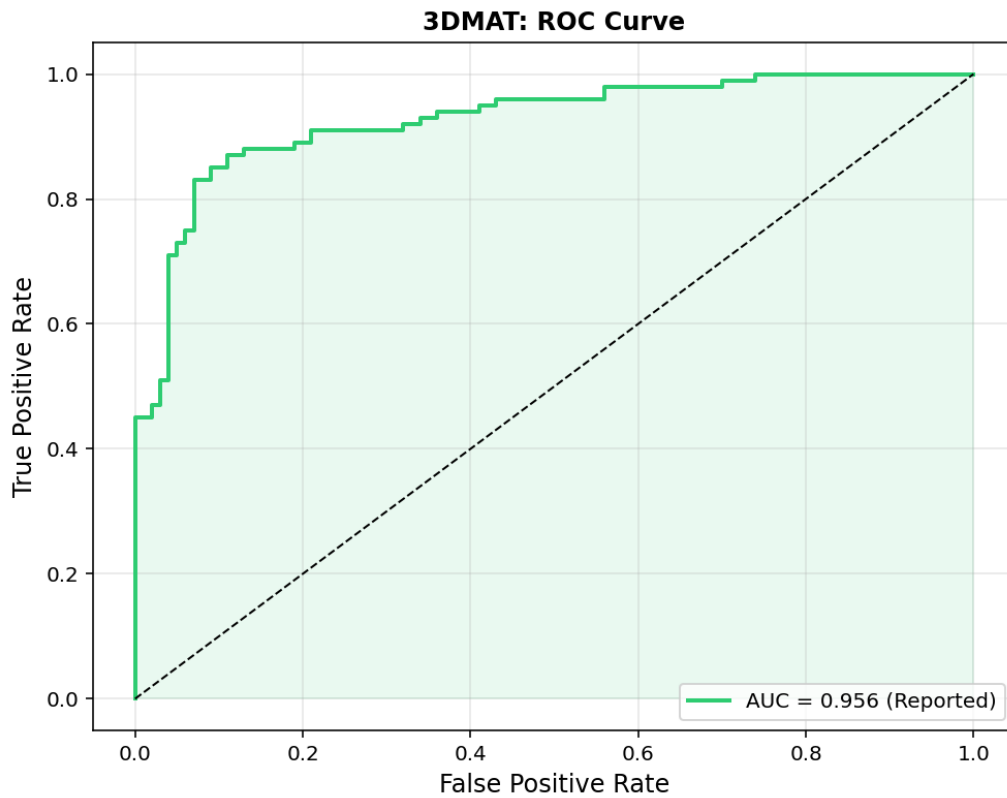
Mean Absolute Error (MAE) = 0.0100

Mean Square Error (MSE) = 0.0001

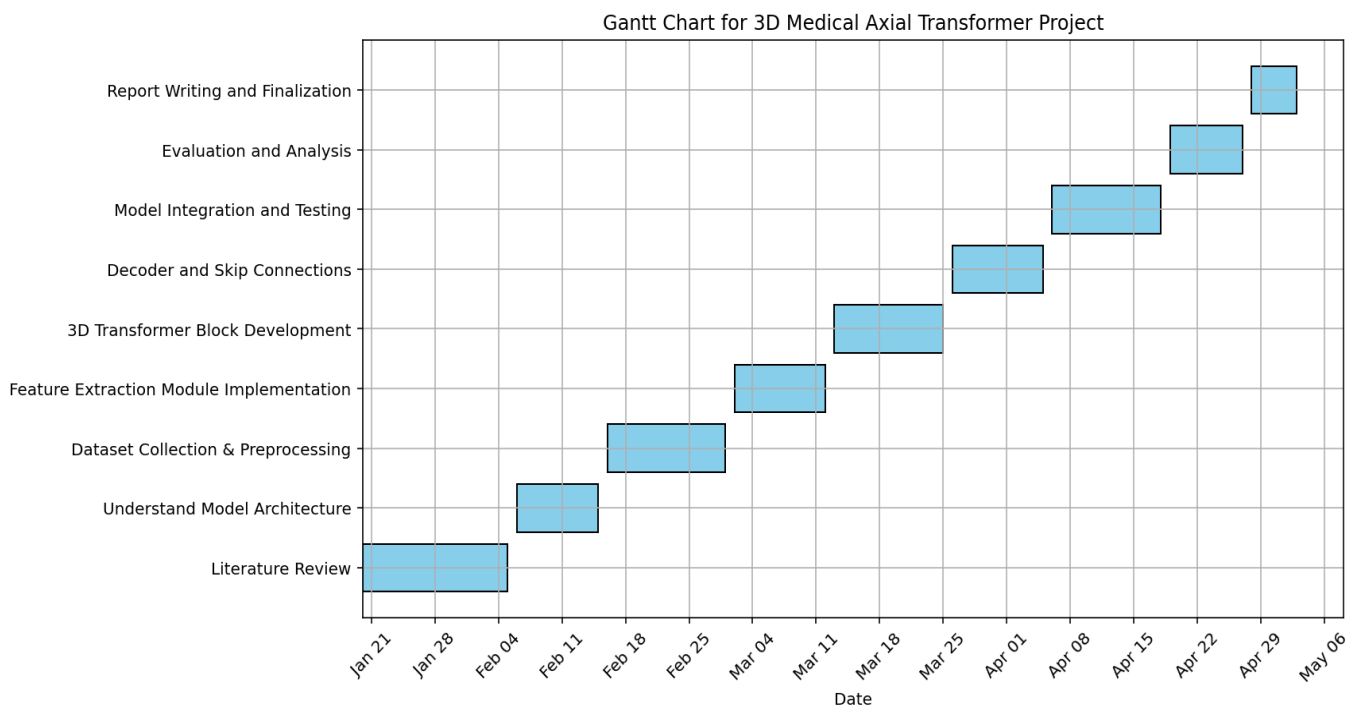
Learning Curve:



ROC Curve:



E-Gantt Chart



References

- [1] Khan Academy. Magnetic resonance imaging (MRI). <https://www.khanacademy.org/test-prep/mcat/physical-processes/proton-nuclear-magnetic-resonance/a/magnetic-resonance-imaging-mri>, n.d.
- [2] R. R. Agravat and M. S. Raval. Brain tumor segmentation and survival prediction. In International MICCAI Brainlesion Workshop, pp. 338–348. Springer, 2019. doi: arXiv:1909.09399v1
- [3] U. Baid et al. The RSNA-ASNR-MICCAI BraTS 2021 benchmark on brain tumor segmentation and radiogenomic classification. arXiv preprint, 2021. doi: arXiv:2107.02314v1
- [4] S. Bakas et al. Segmentation labels and radiomic features for the pre-operative scans of the TCGA-GBM collection. The Cancer Imaging Archive, 2017. doi: 10.7937/K9/TCIA.2017.KLXWJJ1Q
- [5] S. Bakas et al. Segmentation labels and radiomic features for the pre-operative scans of the TCGA-LGG collection. The Cancer Imaging Archive, 2017. doi: 10.7937/K9/TCIA.2017.GJQ7R0EF
- [6] S. Bakas et al. Advancing the Cancer Genome Atlas glioma MRI collections with expert segmentation labels and radiomic features. Scientific Data, 4(1):1–13, 2017. doi: 10.1038/sdata.2017.117
- [7] S. Bakas et al. Identifying the best machine learning algorithms for brain tumor segmentation, progression assessment, and overall survival prediction in the BraTS challenge. arXiv preprint, 2018. doi: arXiv:1811.02629
- [8] A. Berger. Magnetic resonance imaging. BMJ, 2002. doi: 10.1136/bmj.324.7328.35
- [9] Y. LeCun et al. Handwritten digit recognition with a back-propagation network. Morgan Kaufmann, 1990. ISBN: 1558601007
- [10] M. D. C. Preston. Magnetic resonance imaging (MRI) of the brain and spine:

Basics. <https://case.edu/med/neurology/NR/MRI%20Basics.htm>, 2016.

[11] A. Dosovitskiy et al. An image is worth 16x16 words: Transformers for image recognition at scale. arXiv preprint, 2021. doi: arXiv:2010.11929v2

[12] F. Gaillard. Case courtesy of Frank Gaillard, Radiopaedia.org, rID: 27812. <https://radiopaedia.org/articles/glioblastoma-idh-wildtype>, 2023.

[13] A. Hatamizadeh et al. Swin UNETR: Swin transformers for semantic segmentation of brain tumors in MRI images. In International MICCAI Brainlesion Workshop, pp. 272–284. Springer, 2021. doi: arXiv:2201.01266v1

[14] K. He et al. Deep residual learning for image recognition. In Proc. of the IEEE CVPR, pp. 770–778, 2016.

[15] S. Li et al. Medical image segmentation using squeeze-and-expansion transformers. arXiv preprint, 2021. doi: arXiv:2105.09511v3

[16] X. Li et al. Multi-step cascaded networks for brain tumor segmentation. In Brainlesion: Glioma, Multiple Sclerosis, Stroke and Traumatic Brain Injuries, Springer, 2020. doi: arXiv:1908.05887v3

[17] Z. Liu et al. Swin transformer: Hierarchical vision transformer using shifted windows. In Proc. of the IEEE/CVF ICCV, pp. 10012–10022, 2021.

[18] I. Loshchilov and F. Hutter. Decoupled weight decay regularization. Proc. of the IEEE/CVF ICCV, 2015.

[19] Mathworks. Overfitting. <https://it.mathworks.com/discovery/overfitting.html>, n.d.

[20] B. H. Menze et al. The multimodal brain tumor image segmentation benchmark (BraTS). IEEE Trans. on Medical Imaging, 34(10):1993–2024, 2014. doi: 10.1109/TMI.2014.2377694

[21] T. M. Mitchell. Machine Learning. McGraw-Hill, 1997. ISBN: 0070428077

[22] A. Myronenko and A. Hatamizadeh. Robust semantic segmentation of brain tumor regions from 3D MRIs. In Brainlesion: Glioma, Multiple Sclerosis, Stroke and Traumatic Brain Injuries, 2019.

- [22 cont.] FLAIR MRI. <https://mrimaster.com/characterise-image-flair/>, n.d.
- [23] AANS. Brain tumors. <https://www.aans.org/en/Patients/Neurosurgical-Conditions-and-Treatments/Brain-Tumors>, 2023.
- [24] Radiopaedia.org. T1 weighted image. <https://radiopaedia.org/articles/t1-weighted-image>, 2016.
- [25] M. Raghu et al. Do vision transformers see like convolutional neural networks? *NeurIPS*, 34:12116–12128, 2021.
- [26] A. Steiner and A. Kolesnikov. How to train your ViT? Data, augmentation, and regularization in vision transformers. *arXiv preprint*, 2021. doi: arXiv:2106.10270v1
- [27] R. Strudel et al. Segmenter: Transformer for semantic segmentation. In *Proc. of the IEEE/CVF ICCV*, pp. 7262–7272, 2021. doi: arXiv:2105.05633
- [28] A. Vaswani et al. Attention is all you need. *NeurIPS*, 30, 2017.
- [29] F. Wang et al. 3D U-Net based brain tumor segmentation and survival days prediction. In *International MICCAI Brainlesion Workshop*, pp. 131–141. Springer, 2019. doi: arXiv:1909.12901v2
- [30] W. Wang et al. TransBTS: Multimodal brain tumor segmentation using transformer. In *MICCAI 2021*, pp. 109–119. Springer, 2021. doi: arXiv:2103.04430
- [31] H. Ritzema. *Drainage Principles and Applications*. ILRI Publication, 1994.
- [32] K. Clark et al. The Cancer Imaging Archive (TCIA). *J. of Digital Imaging*, 26(6):1045–1057, 2013.
- [33] E. Alberts et al. A nonparametric growth model for brain tumor segmentation in longitudinal MR sequences. In *Brainlesion*, Springer, 2016.
- [34] E. Alberts et al. Uncertainty quantification in brain tumor segmentation using CRFs. In *IEEE ISBI*, pp. 428–431, 2016.
- [35] W. Bidgood et al. Understanding and using DICOM. *JAMIA*, 4(3):199–212, 1997.
- [36] P. A. Yushkevich et al. User-guided 3D active contour segmentation.

Neuroimage, 31(3):1116–1128, 2006.

[37] A. Fedorov et al. 3D Slicer as an image computing platform for the Quantitative Imaging Network. *MRI*, 30(9):1323–1341, 2012.

[38] S. Ourselin et al. Reconstructing a 3D structure from serial histological sections. *Image and Vision Computing*, 19(1):25–31, 2001.

[39] S. Ourselin et al. Robust registration of multi-modal images. In *MICCAI 2002*, Springer.

[40] B. Avants et al. Advanced Normalization Tools: V1.0. July 2009. <http://hdl.handle.net/10380/3113>

[41] J. Juntu et al. Bias field correction for MRI images. In *Computer Recognition Systems*, Springer.